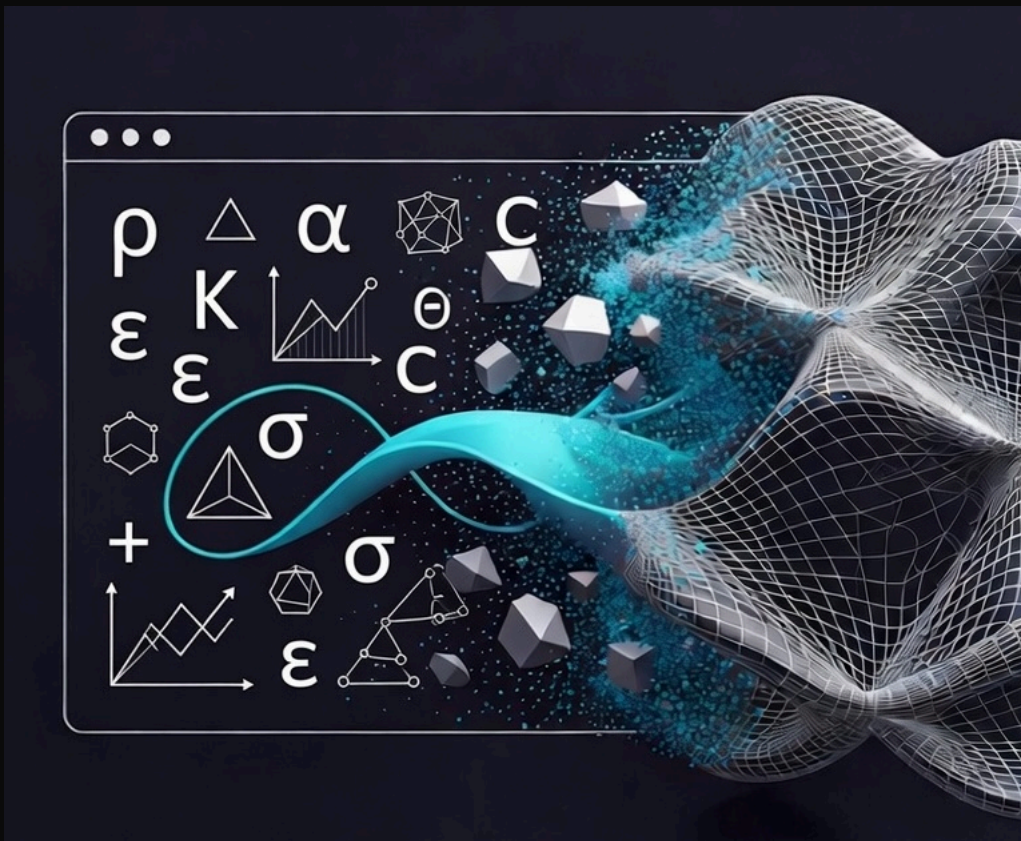




Surrogate Model Training Module (GUI) Manual

The Surrogate Model Training Module is a graphical user interface (GUI) programmed in Python and designed for building and analyzing surrogate models from multidimensional datasets. A surrogate model is a mathematical approximation of a more complex system, allowing for rapid evaluation and analysis of the system's behavior. More details on the surrogate model's mathematical formulation and algorithmic implementation are provided in [Surrogate Model Overview](#).

Developed by Andre Ruybalid and Laurent Capolungo

MST-8 — Los Alamos National Laboratory

BSD-3 License

Version "O# 05190"

October 3, 2026

1. [Introduction](#)
2. [System Requirements and Preconfiguration](#)
3. [Data Management](#)
4. [Configuration](#)
5. [Data Preview](#)
6. [Mesh Operations](#)
7. [Model Training](#)
8. [Results Analysis](#)
9. [Export and Save Options](#)
10. [Additional Notes](#)
11. [Examples](#)
12. [Backend Logic Overview](#)
13. [Troubleshooting](#)
14. [Surrogate Model Overview](#)

Introduction

This Training Module enables users to:

- Load multidimensional datasets
- Configure input and output variables with various scaling options
- Generate and visualize data and interpolation grids (mesh)
- Train interpolation-grid-based surrogate models using the data

- Analyze and visualize results through 2D and 3D plots
- Export results and visualizations

The tool is particularly useful for researchers working with complex simulation data who need to create efficient approximations for further analysis or optimization. The tool was originally designed to train surrogates on polycrystalline data, and is the main tool used for training LAROMance-type surrogates for the MOOSE and BISON frameworks. However, its architecture is generic. It can be applied to any correlated data as long as all arrays are consistent in array length and corresponding indices across arrays represent matching datapoints.

System Requirements and Preconfiguration

To ensure optimal performance and functionality of this application, your system must meet the following software requirements:

This application requires **Python 3.10 or higher** to be installed on your system. Python serves as the core runtime environment for the application. You can verify your Python version by opening a command prompt or terminal and typing `python --version` or `python3 --version`.

Required Dependencies

Package	Purpose	Note
PyQt5	GUI framework (Qt v5 bindings)	
NumPy	Efficient array & numerical operations	
Matplotlib	Plotting & visualisation (integrated with PyQt5)	
SciPy	Advanced scientific computing (optimisation, stats, etc.)	
scikit-learn	Machine-learning utilities (e.g., MinMaxScaler)	

All required dependencies can be installed using pip, Python's package installer, with the following command:

```
pip install PyQt5 numpy matplotlib scipy scikit-learn
```

Launching the Application

The application consists of the **training_module.py** executable and a **lib** directory that holds the core functional methods; you can start it from the command line with:

```
python training_module.py
```

Working Directory

The Surrogate Training Module looks for a working directory located one level up from the surrogate training module, at `../sm_workdir`. This is the place for storing data, and outputs.

The outputs of the Surrogate Training Module are stored in `../sm_workdir/GUIDemo`.

User Interface Overview

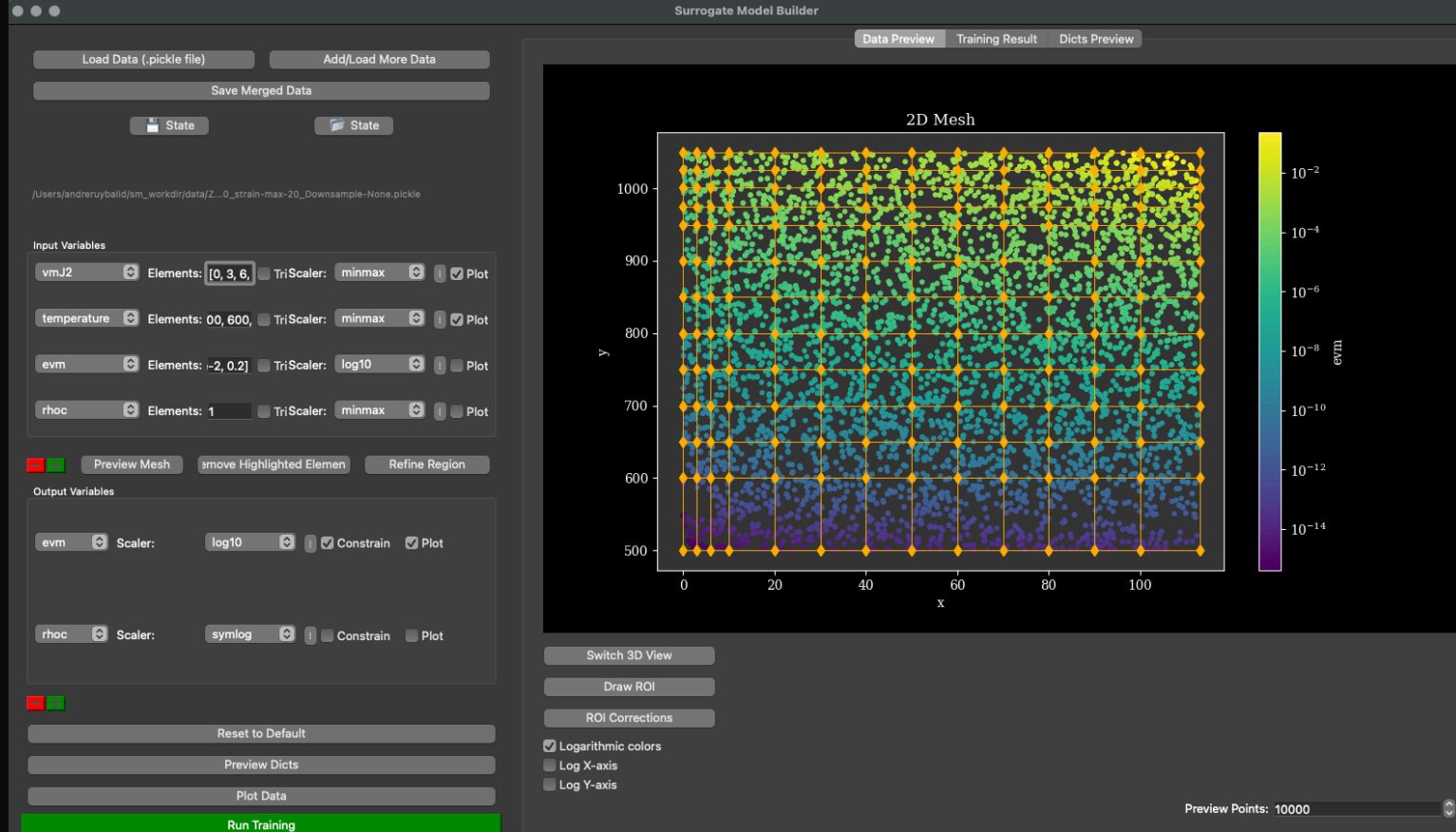
The interface is divided into two main sections:

- **Left Panel:** Configuration settings for model inputs, outputs, and training
- **Right Panel:** Tabbed display area for data preview, training results, and dictionary preview

The application has three main tabs:

1. **Data Preview:** Shows loaded data as 2D/3D plots
2. **Training Result:** Displays surrogate model results after training
3. **Dicts Preview:** Shows configuration dictionaries

Main interface with tab for Data Preview active on the right.



Data Management

Loading Data

The application provides a straightforward interface for importing your datasets, which must be stored in Python pickle format. This section describes the process of loading data into the application and outlines the required data structure for proper functionality.

Follow these steps to import your dataset into the application:

- 1. Locate the Load Button:** At the top of the left panel, you will find the **Load Data (.pickle file)** button. This button initiates the file selection dialog (refer to the screenshot above for the exact location).
- 2. Select Your Pickle File:** Click the button to open a file browser dialog. Navigate to the location of your pickle file and select the file containing your dataset. The application accepts only .pickle or .pkl file extensions.
- 3. Verify Successful Loading:** Once the file is selected and loaded, the complete file path will be displayed below the button. This confirmation indicates that your data has been successfully imported and is ready for analysis.

Required Data Structure

For the application to properly interpret and process your data, the pickle file must contain a dictionary with a specific hierarchical structure. The data must be organized as follows:

```
{
  'data': {
    'var1': [values...], # Input variable 1
    'var2': [values...], # Input variable 2
    ...
    'U': {
      'output1': [values...], # Output variable 1
      'output2': [values...], # Output variable 2
      ...
    }
  }
}
```

All arrays must be consistent: i.e., they must be of the same length, and each index in each array is assumed to correspond to the same data point. In the case of mechanics problems, inputs are typically thermo-mechanical properties and conditions, while outputs are typically mechanical responses, e.g., plastic strain

(rate), or internal state variable rates of change (dislocation density rate of change).

Data Preview

The Data Preview feature allows you to visualize relationships between your input and output variables before proceeding with model training. This functionality is essential for understanding your data distribution, identifying patterns, and verifying data quality. The application provides flexible plotting options with both 2D and 3D visualization capabilities.

After loading data, click the **Plot Data** button to visualize the selected variables.

Plot controls include:

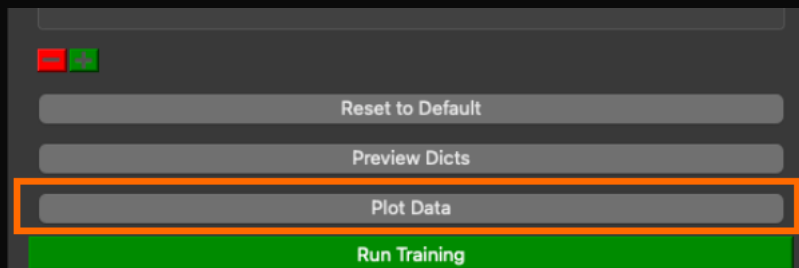
- **Switch 3D View:** Toggle between 2D and 3D visualizations
- **Draw ROI:** Define a Region of Interest by drawing a rectangle on the plot
- **ROI Corrections:** Manually adjust ROI bounds for specific variables
- **Logarithmic colors:** Use logarithmic color scale
- **Log X-axis/Log Y-axis:** Use logarithmic scaling on axes
- **Preview Points:** Control how many data points to display (for large datasets)

Generating a Data Preview Plot

To create a visualization of your loaded data, follow these steps:

1. **Select Variables to Plot:** In the variable list panel, you will see checkboxes labeled "Plot" next to each available variable (both inputs and outputs).
 - Select the checkbox for **at least one input variable**
 - Select the checkbox for **at least one output variable**
 - Optionally, you may select a **second input variable** to create multi-dimensional plots
2. **Generate the Plot:** Once you have selected your variables, click the **"Plot Data"** button. This button is located above the green **"Run Training"** button in the interface.

Plot Data button location.



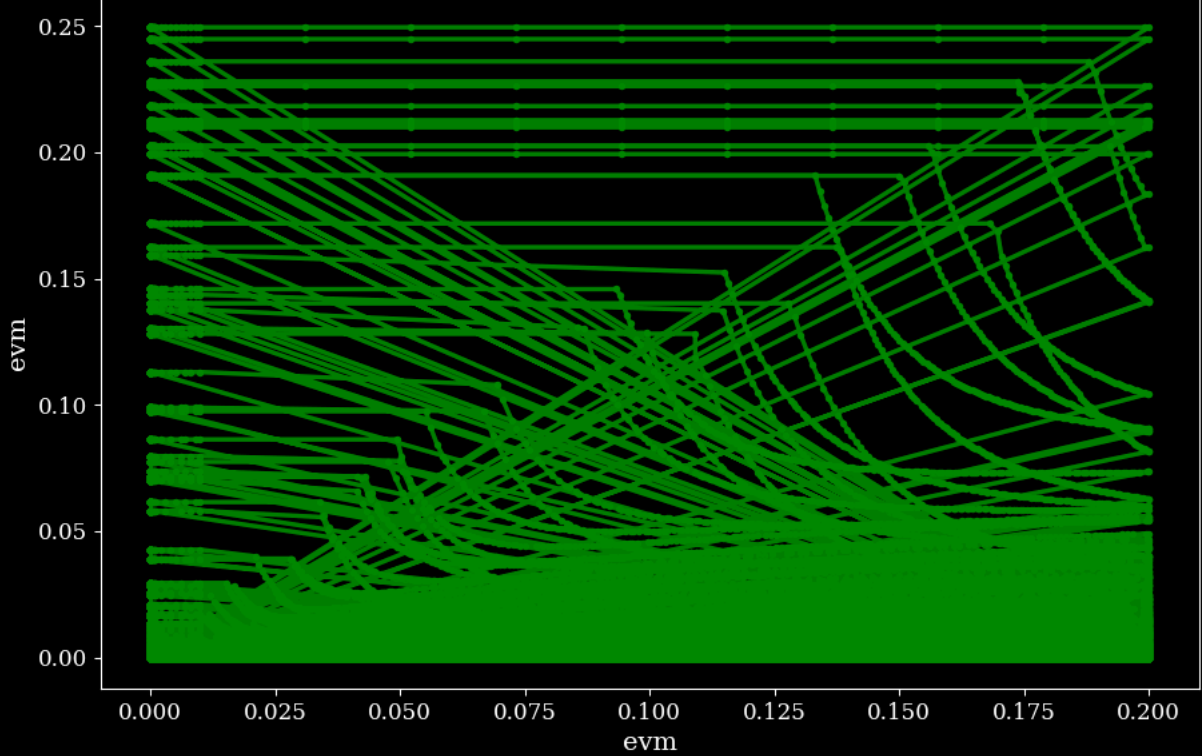
3. **View Results:** The plot will appear in the Data Preview pane, displaying the relationship between your selected variables.

Plot Dimensionality

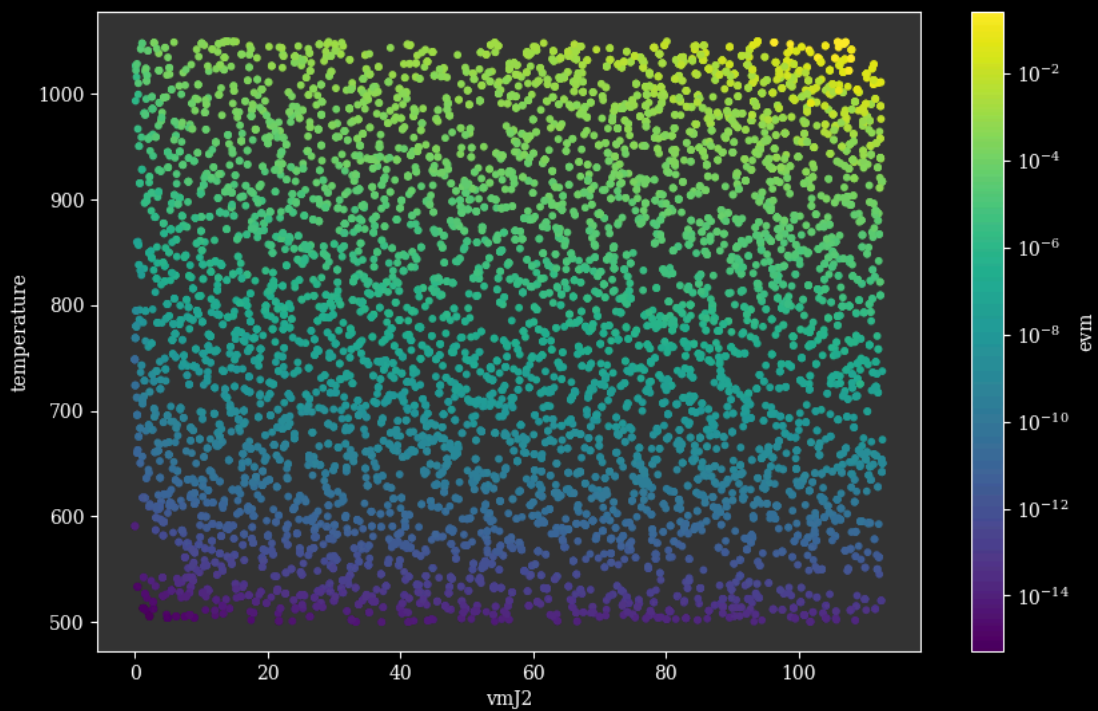
The application automatically adjusts the plot type based on your variable selection:

- **Single Input Variable:** When you select one input and one output variable, the application generates a standard line plot showing the relationship between the two variables.

Data Preview examples with 2D and 3D axes.



- **Two Input Variables:** When you select two input variables along with one output variable, the application creates a visualization showing how the output varies across both input dimensions. You can view this data as either:
 - A **2D scatter plot** with the output represented by color-coded points
 - A **3D scatter plot** with the output displayed on the vertical axis*Data Preview examples with 2D and 3D axes.*



This multi-dimensional plotting capability is particularly useful for understanding complex relationships and interactions between multiple input parameters.

Customizing Plot Settings

After generating a plot using the **Plot Data** button, you can modify various visualization parameters using the controls located below the Data Preview pane. These settings allow you to enhance the clarity and interpretability of your plots.

Plotting options for Data Preview.



Available Plot Settings:

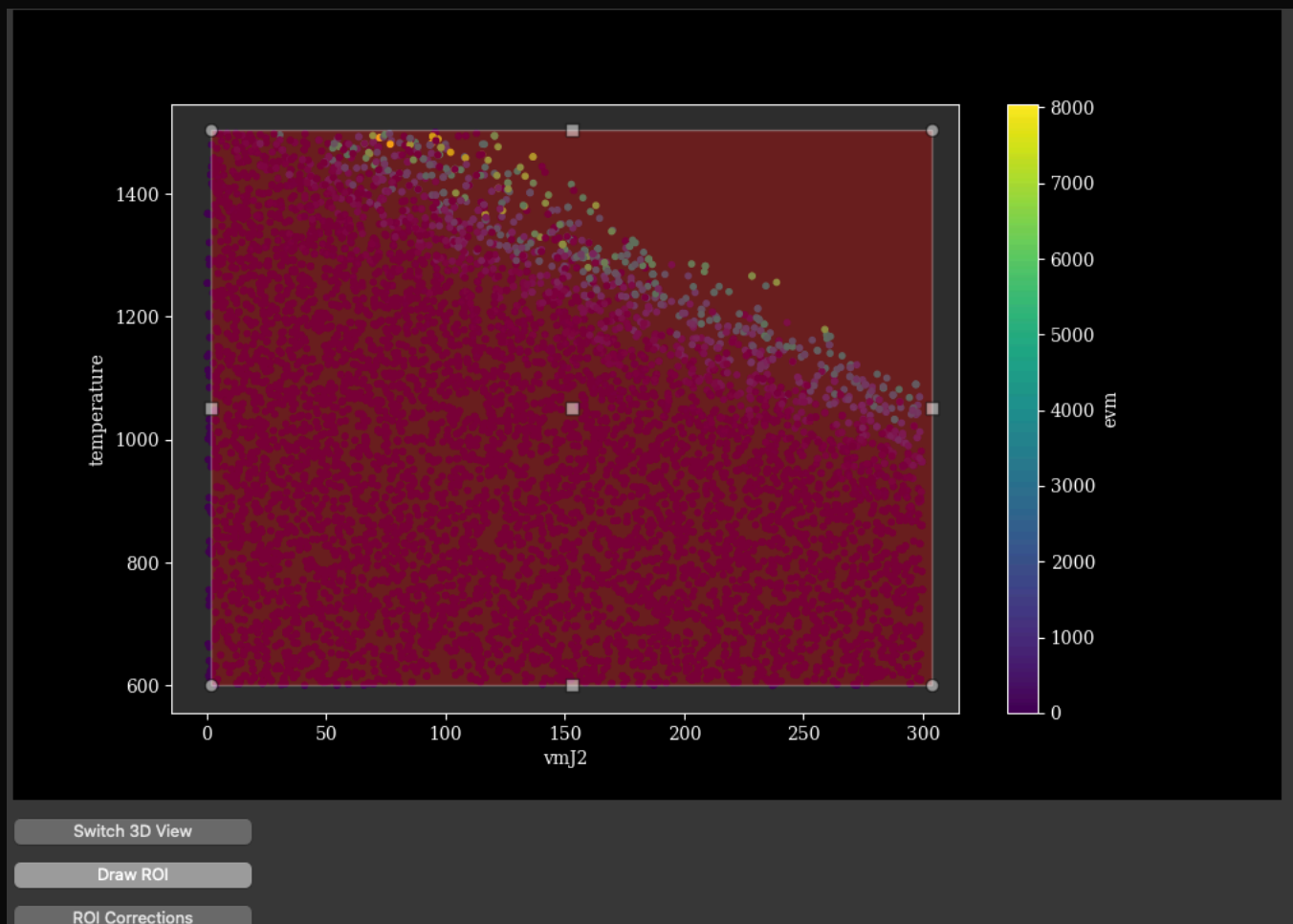
- **Logarithmic Color Scale:** For scatter plots where color represents output values, you can toggle between linear and logarithmic color scaling. A logarithmic scale is particularly useful when your output values span several orders of magnitude, allowing better visualization of both small and large values simultaneously.
- **Logarithmic Axes:** You can independently set each axis (x, y, and z) to use either linear or logarithmic scaling, which is beneficial when data spans multiple orders of magnitude or when emphasizing relative changes.
- **Region of Interest (ROI) Selection**

The application provides tools for defining and refining the Region of Interest (ROI) within your dataset. The ROI determines which subset of your data will be used for subsequent analysis and model training. Upon loading the data, the module automatically adjusts the region of interest to build a surrogate across to the data bounds. There are two ways to manually select or adjust the ROI:

ROI Selection and Adjustment Methods:

1. **Graphical Selection:** Use the interactive drawing tools (Draw ROI) and buttons to directly select regions on the plot itself. This visual approach is intuitive and allows you to identify interesting features or exclude outliers by simply drawing on the visualization.

Training Module: Region of Interest Configuration.



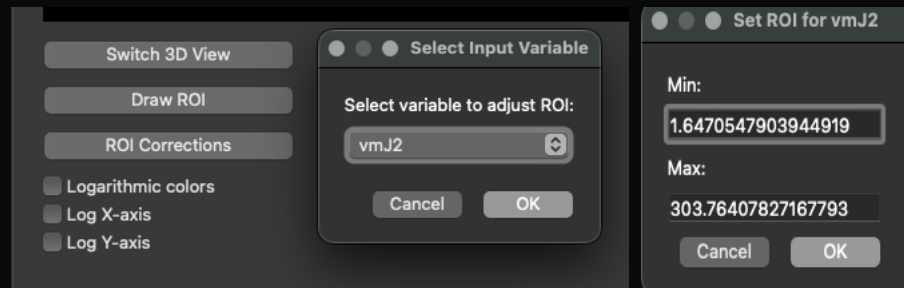
The ROI selection will be reflected in real-time on your plot, typically displayed as a highlighted region or bounding box, making it easy to verify your selection before proceeding with training.

After drawing the ROI, "unselect" the "Draw ROI" button, single-clicking it again, to get out of the ROI drawing mode. If you don't "unclick" the "ROI Draw" button, then you are at risk of changing the ROI upon the next click (for example when refining the mesh).

Finally, proceed to configuring and previewing the mesh.

- Numerical Input:** Enter specific numerical bounds for each variable using the input fields in the menu below the Data Preview pane. This method provides precise control when you know the exact range of interest.

ROI Correction Pop-Up Menu's.

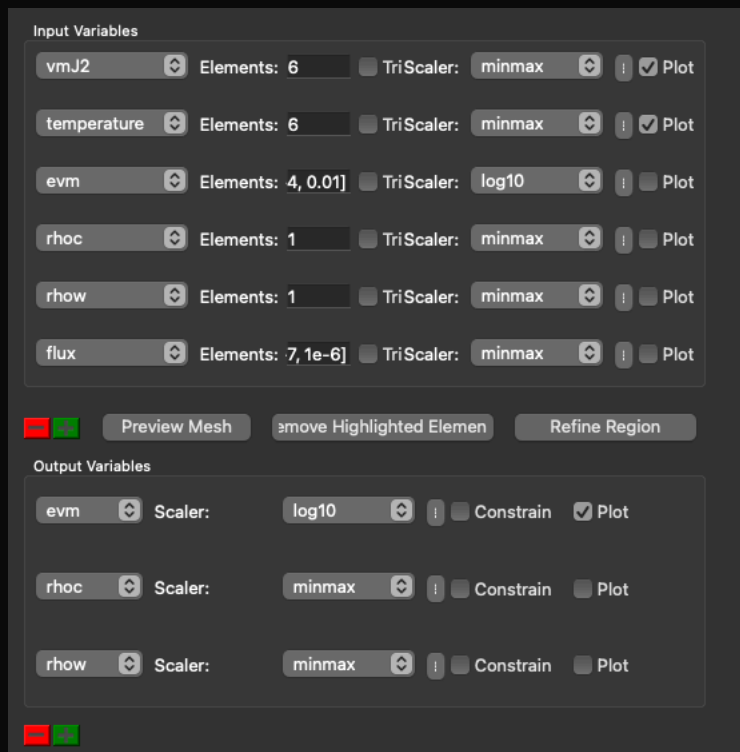


Both methods can be used in combination, allowing you to start with a rough graphical selection and then fine-tune the boundaries with numerical precision.

Configuration

The configurations for training the FEM-inspired surrogate on the left of the interface are grouped in terms of Input variables and Output variables (below options for Data Loading). This is where the training configurations for the surrogate are set, in terms of mesh design (nodal placement) and data scaling. The main variable (I/O) selection is done in the following menu, and will be explained below:

Training Module Input/Output selection.



Input Variables

Input variables represent the independent variables of your dataset (e.g., temperature, pressure, etc.).

To configure input variables:

1. Use the + and - buttons below "Input Variables" to add or remove variables.
2. For each input variable:
 - Select the variable name from the dropdown
 - Set the number of elements for mesh generation
 - Check "Tri" if you want triangular elements (for 2D variables)
 - Check "Plot" to select this variable for visualization (max 2)
 - Choose a scaling method (minmax, log10, compress, symlog)
 - Click the  button to configure scaling parameters

Output Variables

Output variables represent the dependent variables (results) you want to model.

To configure output variables:

1. Use the + and - buttons below "Output Variables" to add or remove variables.
2. For each output variable:
 - Select the variable name from the dropdown
 - Select a scaling method (minmax, log10, compress, symlog)
 - Click the  button to configure scaling parameters
 - Check "Constrain" to set lower/upper bounds for the output
 - Check "Plot" to select this variable for visualization

Scaler Settings

Scalers are essential because the raw input and output variables often differ by several orders of magnitude, which can severely ill-condition the normal-equation matrix (see [Surrogate Model Overview](#)) solved during training. By normalizing all variables to a common scale: typically a unit interval or a symmetric range like [-1, 1], the linear system becomes better conditioned, leading to more stable and accurate least-squares solutions and faster convergence of the underlying sparse solvers. The scalers can be selected from the dropdown menu. Upon selecting a scaler, a separate menu pops up with scaler configurations (ranges, and scaling factors), as is seen in the following screenshot:

Training Module: Scaler Configuration Menu



Clicking the  button (to the right of the scaler dropdown menu's of each variable) to opens a dialog to configure scaling parameters. The scaling of input and output variables is performed by the backend functions defined in [lib/sm_data.py](#). The primary function responsible for constructing the scalers is `fit_and_transform_input`. It builds a dictionary of scaler objects for inputs and outputs based on the user-selected scaling type.

Below is a detailed mathematical description of each scaling option and the corresponding implementation in the code:

- **minmax** – Min-Max scaling

- **Mathematics:** For a variable (x), the scaled value is

$$x' = a + \frac{(x - x_{\min})}{x_{\max} - x_{\min}} \times (b - a)$$

where ($[a,b]$) is the target range (default $[-1, 1]$).

- **Implementation:** In [lib/sm_data.py](#) the function creates an instance of `sklearn.preprocessing.MinMaxScaler(feature_range=(a, b))` and calls `fit_transform` on the input matrix.

- **log10** – Log-base-10 scaling

- **Mathematics:**

$$x' = \log_{10}(\text{factor} \times (x - \text{lowerbound}))$$

Values are first shifted by Lowerbound to ensure positivity, multiplied by factor (a user-defined scaling factor), and then the base-10 logarithm is taken.

- **Implementation:** The scaler is built as a custom lambda/function in **lib/sm_data.py** that applies `np.log10(np.clip(factor * (x - lowerbound), a_min=1e-12, a_max=None))`. The inverse transform (used when exporting results) applies the exponential back-conversion.

- **compress** – *Logarithmic compression*

- **Mathematics:** A common form is the *log-compress* transform

$$x' = \frac{\log(1 + c |x|)}{c}$$

where (c) is the *compressor* factor supplied by the user. This compresses large magnitude values while preserving sign.

- **Implementation:** In **lib/sm_data.py** a custom CompressScaler class (or a pair of forward/backward functions) implements the above formula using `np.log1p` for numerical stability.

- **symlog** – *Symmetric log scaling*

- **Mathematics:**

$$x' = \text{sign}(x) \log_{10} \left(1 + \frac{|x|}{\text{base}} \right)$$

The base (often called the *linear region width*) is defined by the lowerbound/upperbound parameters, controlling the transition between linear and logarithmic behaviour around zero.

- **Implementation:** **lib/sm_data.py** provides a SymLogScaler that applies the above expression element-wise, with clipping to avoid division by zero. The inverse transform restores the original scale using the exponential.

All scalers are stored in a dictionary returned by the scaler-creation routine, e.g.:

```
scalers = {
    'input': input_scaler,    # MinMaxScaler / custom lambda depending on selection
    'output': output_scaler  # Same type as input or independent choice
}
```

During model training the data are passed through `scalers['input'].fit_transform(X)` and `scalers['output'].fit_transform(y)`. When the surrogate is later used for prediction, the inverse transforms (`inverse_transform`) are applied to map the surrogate output back to physical units.

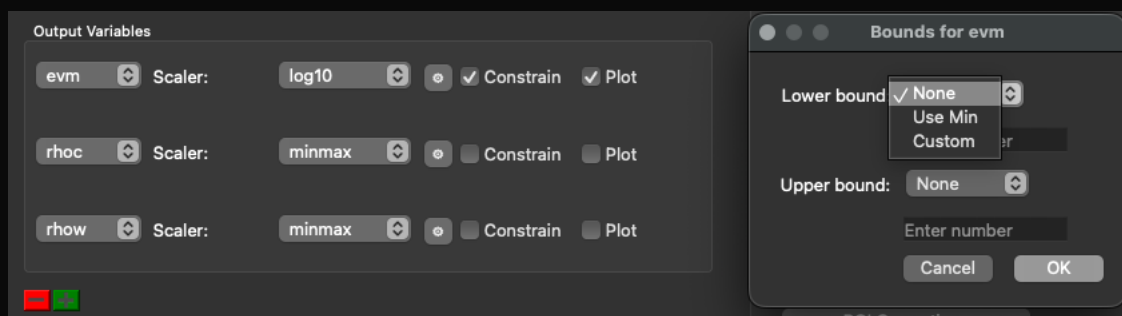
Custom scalers can be implemented adjacent to the existing scalers, following the same class definitions as inspired by standard sk-learn scalers. The scalers will have to be added to the **lib/sm_data.py** script as a class, using the format of sk-learn scaler definitions. Then, the scalers need to be added to the dropdown selection menu of the GUI code in **training_module.py**.

Constraint Settings

When the **Constrain** checkbox is enabled for an output variable, a dialog appears allowing the user to set lower and upper bounds for that variable. These bounds are enforced during model training to keep surrogate predictions within physically realistic limits.

The following menu pops up when the **Constrain** checkbox is checked:

Training Module: Constraint Menu



Usage in Training Module

- **“Use Min” / “Use Max”** – The GUI automatically expands these options into the 10 % safety-margin bounds described above, allowing a modest amount of extrapolation while still enforcing realism.
- **Leave a bound empty** – Selecting **None** (or leaving the field blank) removes the constraint on that side, effectively using $-\infty$ or $+\infty$.
- **Custom numeric bound** – Entering a specific number fixes a hard limit; the optimizer will never produce nodal values (and thus surrogate predictions) outside that interval.

How the constraints are applied in the backend logic

In the training routine `sm_train` (found in `lib/sm_build.py`) the following process is used:

1. **Gathering user-defined limits** – The dictionary `args["constrain_regression"]` is built when the user interacts with the Training Module, and contains, for each output variable, optional lowerbound and upperbound entries. If a variable is not listed, no constraint is applied.
2. **Interpreting special keywords** – The strings "min" and "max" act as shortcuts. "min" is replaced by the smallest observed value of the output (reduced by 10% as a safety margin), while "max" is replaced by the largest observed value (increased by 10%). If a bound is None, it is treated as unbounded ($-\infty$ or $+\infty$).
3. **Performing a bounded regression** – The linear system that determines nodal values ($M \cdot w = rhs$) is solved with `scipy.optimize.lsqr_linear`. This routine solves a least-squares problem while respecting the provided box constraints, yielding a **constrained linear regression**.
4. **Storing the applied bounds** – After the solve, the actual lower and upper limits used are saved in the surrogate model dictionary (`SM['constrained'][key]`) for later reference or export.

These constraint options help guarantee that model outputs stay within physically plausible ranges, such as non-negative values.

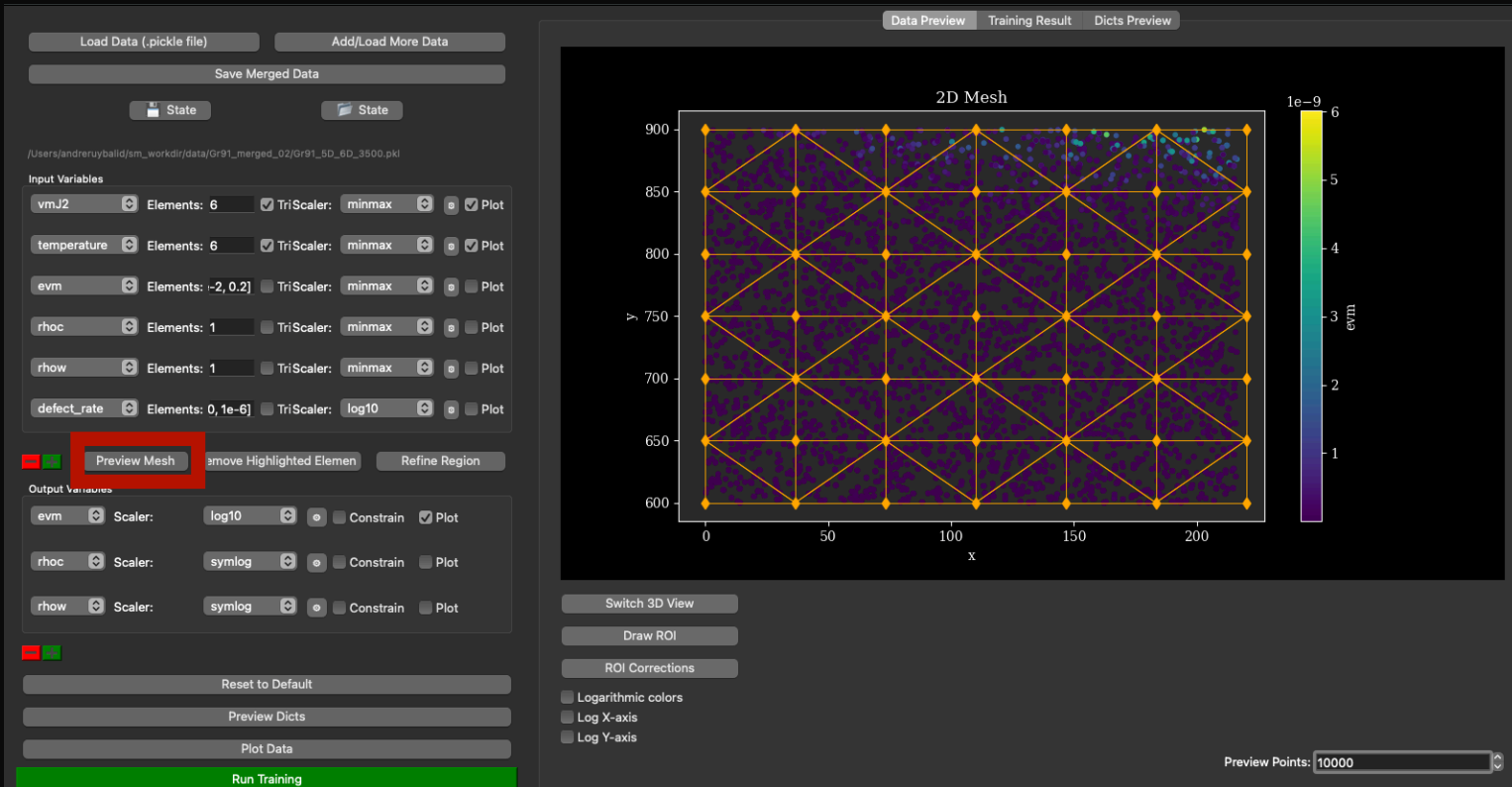
Interpolation Grid Design (Mesh Building)

The **Mesh Preview** step is one of the most sophisticated features of the Surrogate Model Training Module. It translates the high-dimensional input space into a finite-element-style interpolation mesh that the training algorithm can use. The preview is generated on-the-fly by the GUI (`training_module.py`) which calls the low-level shape-function and mesh generators in `lib/fe_shapes.py`.

Workflow Overview

1. **Load data** and define an ROI (or accept the automatically inferred one).
2. **Select the "Tri" option** for the relevant input variables to use triangular elements; this selection is required for the later refinement method to work properly.
3. Click **Preview Mesh** – the GUI runs the steps above, updates `self.preview_nodes` and `self.preview_conn`, and draws the mesh on the plot.
4. Inspect the overlay: dense regions appear with many small elements, while sparse regions are highlighted.
5. If necessary, use **Refine Region** or **Remove Highlighted Elements** to adjust the mesh before proceeding to training.

Training Module: Mesh Preview



What the preview does behind the scenes

1. **Select two base dimensions** – The GUI extracts the first two input variables that have a defined element count in `mesh_specs['element_numbers']`. These become the *base 2-D plane* for visualization.
2. **Build a 2-D base mesh** –

- If `mesh_specs['tri_elements']` contains a dimension name, `fe_shapes.build_simplex_extrusion_nd` constructs a Delaunay-triangulated simplex mesh (triangles).
 - Otherwise `fe_shapes.build_hypercube_mesh` creates a regular rectangular grid.
- Both functions use **custom Legendre-polynomial-based shape functions** that give optimal approximation properties while keeping the degrees-of-freedom (DOF) minimal.

- Extrude to higher dimensions** – For problems with more than two inputs, the base mesh is **extruded** along the remaining axes. The extrusion routine appends extra coordinate columns and copies the 2-D element connectivity, producing *prism-like* elements (triangular prisms or hexahedral bricks in 3D and higher dimensional shapes beyond 3D) that faithfully represent the full ND domain.
- Compute element density** – Immediately after mesh generation the routine `fe_shapes.data_density_in_element_2d*` evaluates how many training points fall inside each element. Elements that are under-populated are highlighted in the preview, guiding the user toward refinement or removal.
- Display the mesh** – The GUI hands the node (`preview_nodes`) and connectivity (`preview_conn`) arrays to `lib/sm_plot.visualize_mesh`, which draws the mesh overlay on the data-scatter plot. Nodes are shown as small markers, element edges as light lines, and optionally element numbers can be displayed for debugging.

Why this matters

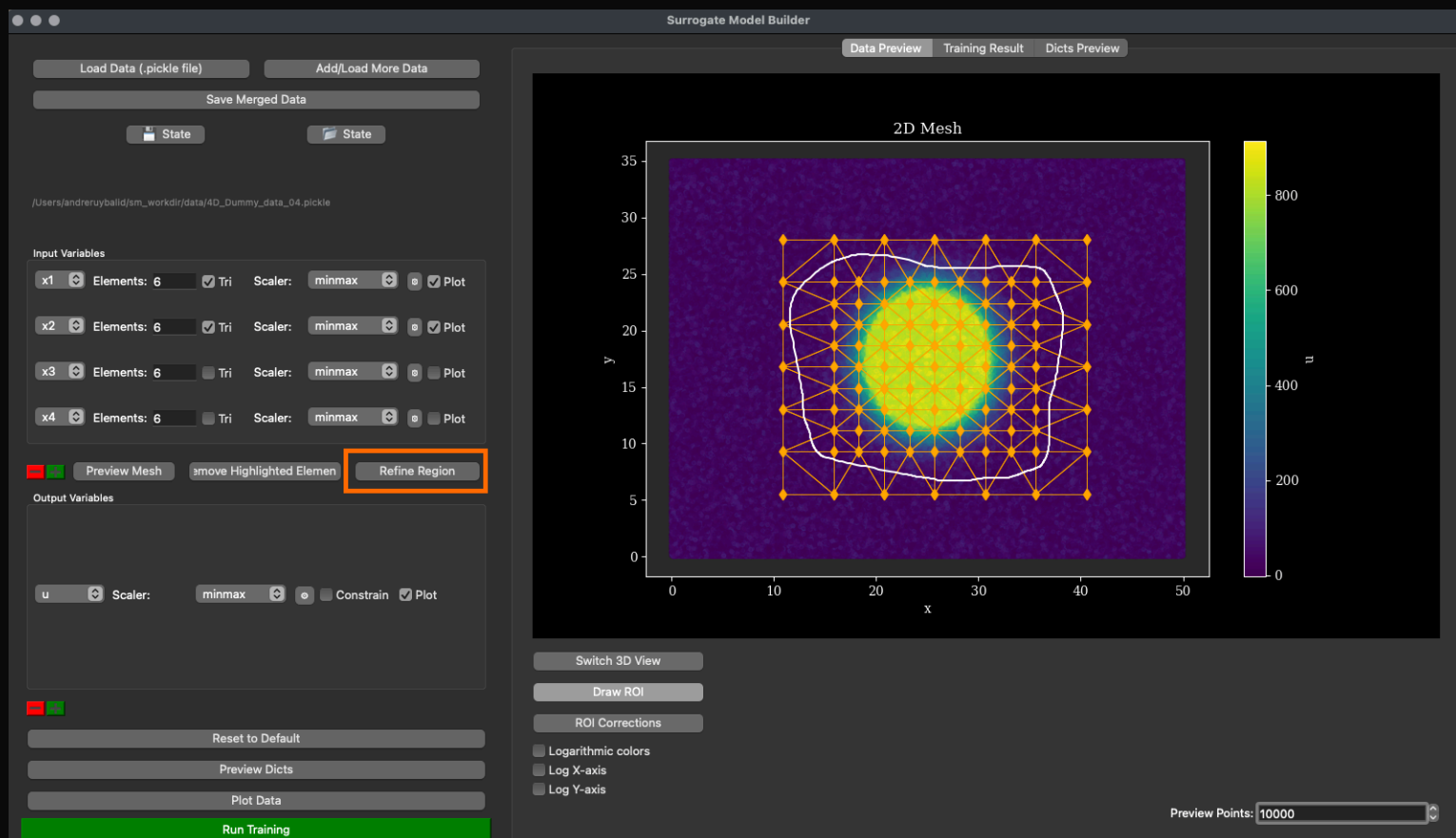
- **Higher-dimensional extrusion** – Extending a 2-D simplex or rectangle into 3-D, 4-D, or even 8-D requires careful bookkeeping of node ordering and connectivity. The code automatically generates the correct connectivity tensors while preserving the orientation of each element.
- **Minimal DOF, maximal flexibility** – By using extruded simplices or hypercubes the mesh can be refined locally (via the *Refine Region* tool) without exploding the total node count. The algorithm balances mesh resolution against computational cost, making it scalable to many-dimensional problems.

This preview gives immediate visual feedback on how the surrogate will discretize the input space, in light of the data (distribution and quality), allowing you to fine-tune mesh density, element type, and dimensional extrusion before committing to a surrogate training run.

Mesh Refinement

The **Refine Region** feature provides an advanced, graphics-based way to locally increase mesh resolution directly from the data preview pane. **Typical workflow**

1. Load your dataset and generate an initial mesh via **Preview Mesh**.
2. Inspect the data preview and identify regions where the surrogate error is large or where physical phenomena change rapidly.
3. Click **Refine Region**, draw a lasso around the target area, and press **Apply** (or the corresponding GUI button).
4. The mesh is instantly updated; you can re-preview to confirm the additional nodes and element density.
5. Proceed to **Run Training** – the refined mesh will be used for building the surrogate, yielding higher fidelity in the selected zones.



Training Module: Mesh Refinement. The white lasso line in the "Data Preview" becomes visible after clicking "Refine Mesh" button and a shape is drawn while clicking in the data field.

How it works

Internally it invokes the `refine_lasso_region` routine in `lib/fe_shapes.py`, which implements a flexible 2-D lasso-based refinement algorithm that works even for meshes that have been extruded into higher dimensions.

- User interaction** – In the GUI the user clicks the **Refine Region** button, then draws a free-hand polygon (lasso) on the data preview plot to mark the area that needs higher resolution.
- Polygon to data space** – The coordinates of the lasso vertices (`verts`) are transformed from screen pixels to the underlying physical coordinate system of the mesh.
- Node selection** – The routine examines `preview_nodes`, the current set of mesh node coordinates, and determines which nodes lie inside the polygon.
- Edge detection & splitting** – For every mesh edge that crosses the lasso boundary, a new node is inserted at the midpoint. This creates additional degrees of freedom exactly where the user has indicated.
- Extrusion handling** – If the mesh is extruded (i.e., the simulation is higher-dimensional), `refine_lasso_region` calls the helper `refine_lasso_region_2d` on each extruded slice, ensuring that refinement is applied consistently across all dimensions.
- Connectivity update** – After new nodes are generated, the element connectivity matrix is rebuilt so that the refined region integrates seamlessly with the original mesh while preserving element topology.
- Precision control** – The `decimals` parameter controls the rounding of newly created node coordinates to avoid floating-point noise, and the `spacing` parameter allows the user to set a minimal distance between inserted nodes for very dense refinements.

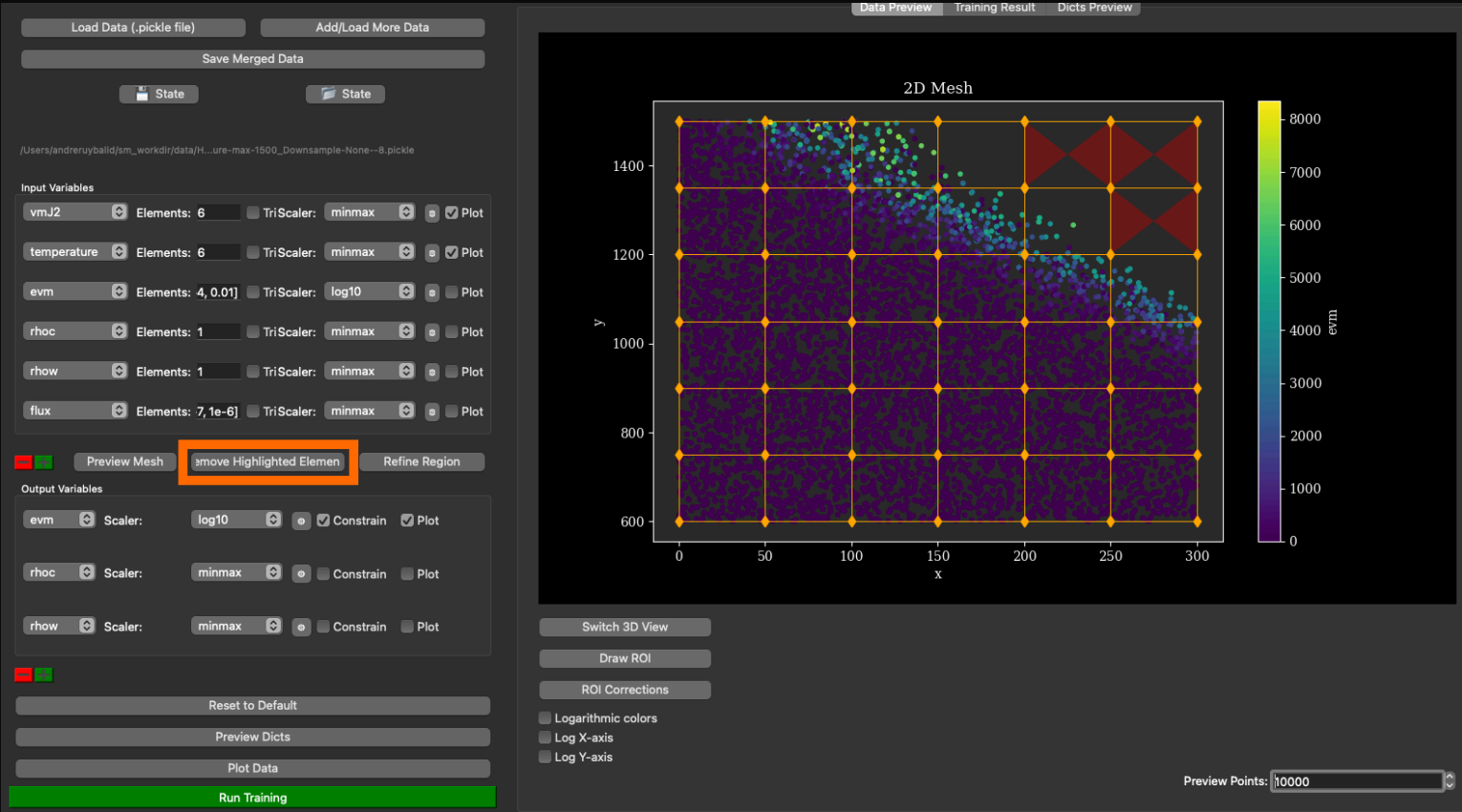
Why it matters

- Targeted resolution** – By refining only the regions of interest (e.g., where data density is high or physical gradients are steep), the surrogate model can capture fine-scale features without exploding the overall mesh size.
- Interactive workflow** – Users can see the effect of refinement immediately on the preview plot, iterate quickly, and fine-tune the mesh geometry before committing to a full training run.
- Versatility** – The same lasso-based approach works for both 2-D base meshes and higher-dimensional extruded meshes, making it a powerful tool for multi-parameter surrogate modeling.

This capability elevates the meshing process from a static, uniform grid to a dynamic, user-driven refinement step, giving the user precise control over where computational effort and resolution is focused.

Remove Elements

Training Module: Empty Element Handling

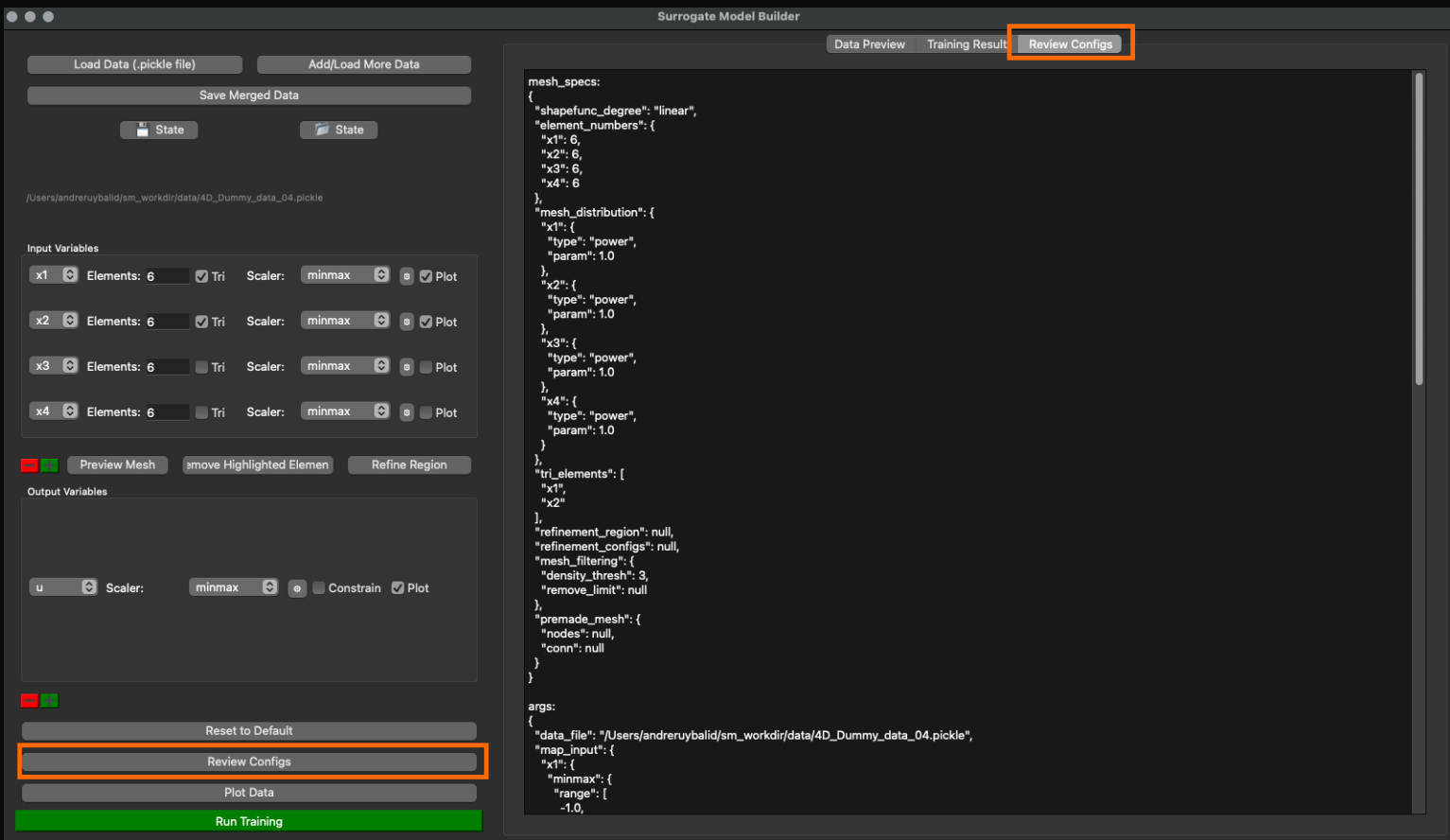


To remove elements with low data density:

1. Preview the mesh to highlight elements with insufficient data points.
2. Click **Remove Highlighted Elements** to clean the mesh.

Review Configurations

All of the above configurations set by the Training Module are stored in a dictionary, which can be reviewed in the right-hand tab "Review Configs" and updates when the button "Review Configs" is clicked, as indicated in the screenshot below:



Training Module: Review Configurations

The dictionaries displayed in the **Review Configs** tab — `mesh_specs`, `args`, and `roi` — along with the training data (DATA), constitute the primary input arguments for the main training function. This function is invoked in the backend when the **Run Training** button is pressed after all configurations have been set (see the next section of this manual).

Mesh Specs (`mesh_specs`)

Key	Type	Description
shapefunc_degree	string	Polynomial degree of shape functions (e.g., "linear").
element_numbers	dict	Number of elements per dimension (x1, x2, ...).
mesh_distribution	dict of dicts	Distribution type ("power" etc.) and parameter for each dimension.
tri_elements	list	Dimensions that use triangular elements.
refinement_region	null or dict	Region to be refined; null means no refinement.
refinement_configs	null or dict	Additional refinement settings.
mesh_filtering	dict	Controls density filtering (density_thresh, remove_limit).
premade_mesh	dict	Pre-built mesh with nodes (list of coordinates) and conn (connectivity).

Training Arguments (`args`)

Key	Type	Description
data_file	string	Path to the pickle file containing training data.

Key	Type	Description
map_input	dict	Per-input scaling (e.g., minmax range) for each variable.
map_output	dict	Scaling for output variables (e.g., u).
constrain_regression	dict	Optional bounds for regression; null means none.
output_directory	string	Directory where training results are written.
deconcat	bool	Whether to de-concatenate multi-output data.
plot_histograms	bool	Plot input/output histograms after loading.
debug	bool	Enable debug output.
remove_sparse_elements	bool	Remove mesh elements with insufficient data density.
sparse	bool	Use sparse matrix formulation for training.

Region of Interest (roi)

A dictionary with keys representing each input variable name (string) and the values represent the minimum and maximum values in a tuple format, e.g., for four inputs (x1-x4):

Variable	Range (min, max)
x1	[10.864962390921633, 40.58147162160486]
x2	[5.519615700273317, 28.05018118866506]
x3	[6.666780747902234e-06, 14.999989831773672]
x4	[5.516584646782463e-06, 9.999993833611697]

Model Training

Loading Training Results (Optional: only when training results are available)

The **Load Training** button (orange) allows you to import a previously saved training session (generated by the **Export** feature or by a prior run). This enables immediate inspection and plotting of results without retraining.

Load Training Results

Load Data (.pickle file)

Add/Load More Data

Save Merged Data

State

State

/Users/andreruyballd/sm_workdir/data/G...r/F4Augmented_F3post_lrr_decon.pickle

Input Variables

vmJ2

Elements: 1

TriScaler: minmax

Plot

temperature

Elements: 1

TriScaler: minmax

Plot

evm

Elements: 1

TriScaler: log10

Plot

rhoc

Elements: 1

TriScaler: minmax

Plot

rhov

Elements: 1

TriScaler: minmax

Plot

dloops_density_values

Elements: 1

TriScaler: minmax

Plot

dloops_size_values

Elements: 1

TriScaler: minmax

Plot

bd_density_values

Elements: 1

TriScaler: minmax

Plot

Preview Mesh

Remove Highlighted Element

Refine Region

Output Variables

evm

Scaler: log10

Constrain

Plot

rhoc

Scaler: symlog

Constrain

Plot

rhov

Scaler: symlog

Constrain

Plot

Reset to Default

Preview Dicts

Plot Data

Run Training

Data Preview

Training Result

Dicts Preview

Load Training

2D View

3D View

Nodal PCA

Plot Time Series (beta)

Show Data Points

Show TriSurf

Transform Outputs

Show Nodes

Colormap: hot

Export

Dark Theme

3D Points: 200

In order to load a training result, the corresponding data file should be loaded first (the same dataset as was used originally during training. This information can be found in the stored "SM.pickle" file that is output by the Module upon successful training). After loading data, the input/output variables selected for plotting the training results should be selected in the left-hand Module pane.

Functional Steps

1. **Open File Dialog** – When clicked, the Training Module prompts you to select a *.pickle file containing a saved training dictionary.
2. **Deserialize Data** – The selected file is opened in binary mode and loaded with `pickle.load`. The dictionary typically holds:
 - Mesh nodes and connectivity (`preview_nodes`, `preview_conn`)
 - Nodal output coefficients (w or similar)
 - Scaling information and variable lists
 - Pre-computed result arrays for quick plotting
3. **Update Internal State** – The method assigns the loaded objects to the application's attributes (`self.preview_nodes`, `self.preview_conn`, `self.result_dict`, etc.) and restores the ROI, mesh specifications, and variable selections.
4. **Refresh GUI Elements** –
 - The mesh preview is regenerated via `self.preview_mesh` utilities.
 - Result plots are rebuilt by calling the same routines used after training (`self.update_result_plot`, `self.show_prev_plot`, etc.).
 - All toggle checkboxes (Data Points, TriSurf, Nodes, etc.) reflect the state stored in the loaded session.
5. **Enable Interaction** – With the loaded data now present, the user can immediately:
 - Switch between 2-D and 3-D result views.
 - Use the PCA, Time-Series, and export buttons.
 - Adjust colormap, scaling, and other visual options.
6. **Error Handling** – If the file cannot be read or is missing required keys, a warning dialog informs the user and aborts the load operation.

By following these steps, the **Load Training** functionality provides a fast workflow for revisiting past training runs, comparing different models, or sharing results with collaborators.

The training process:

- Generates a mesh based on your configuration
- Maps data to the mesh
- Calculates nodal values
- Creates the surrogate model

Running Training

1. Configure input and output variables, and scalars.
2. Define an ROI .
3. Click the green **Run Training** button to start the training process.

Calling the **Run Training** button ultimately triggers the most critical backend function in the workflow: `smb.sm_train`. The dictionaries shown in the **Review Configs** tab (`mesh_specs` and `args`) together with the training data (`DATA`) provide the essential input arguments for this function:

```
train_output, SM = smb.sm_train(roi, mesh_specs, args, DATA)
```

During the training process, some information will be printed to the console, including:

1. Some basic data information, including Region of Interest, and number of datapoints
2. Specifics about the surrogate model, including number of degrees of freedom
3. The step in the calibration (training) process
4. Possible missing data points in specific regimes
5. The R-squared value problem conditioning (once training is completed)

For example:

```
Approximated response computed.  
R-squared u:      0.85  
Condition number M: 6707.29
```

Training Parameters

Important parameters that affect training include:

- Element numbers per dimension
- Triangular vs rectangular elements

- Data scaling methods
- Mesh refinement regions
- Output constraints

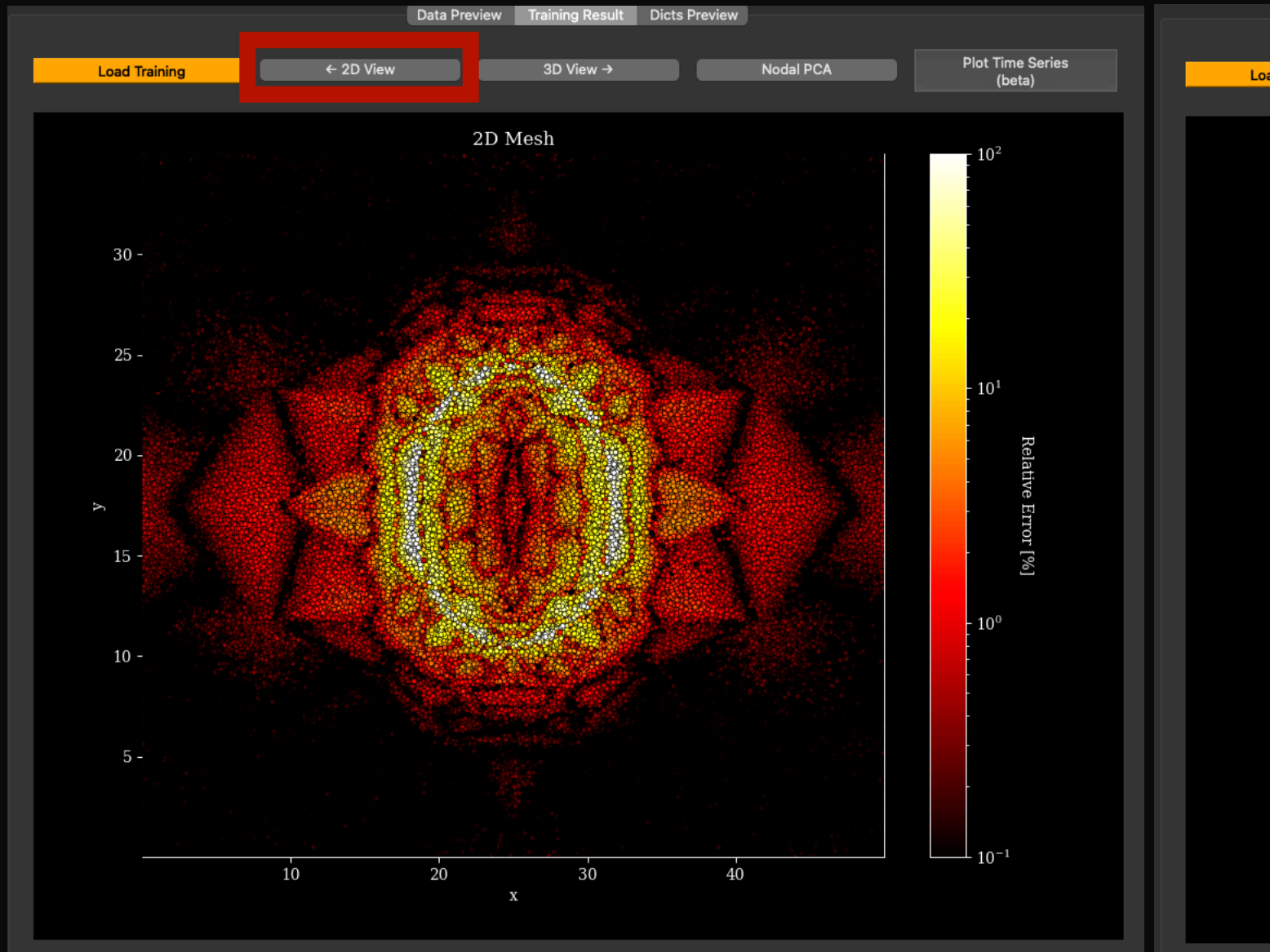
Results Analysis

Results Visualization

After training, results are automatically displayed in the "Training Result" tab with:

- 2D plot showing relative error in a color map, which can be selected in the visualization controls (see below).
- 3D surface plot showing the surrogate model and training data. Here, the ● red data points represent the reference data, and the ■ black squares represent the surrogate calculation.

Use the **← 2D View** and **3D View →** buttons to switch between views.



Load Training Results. The example shown here is a result from the 4D example dataset.

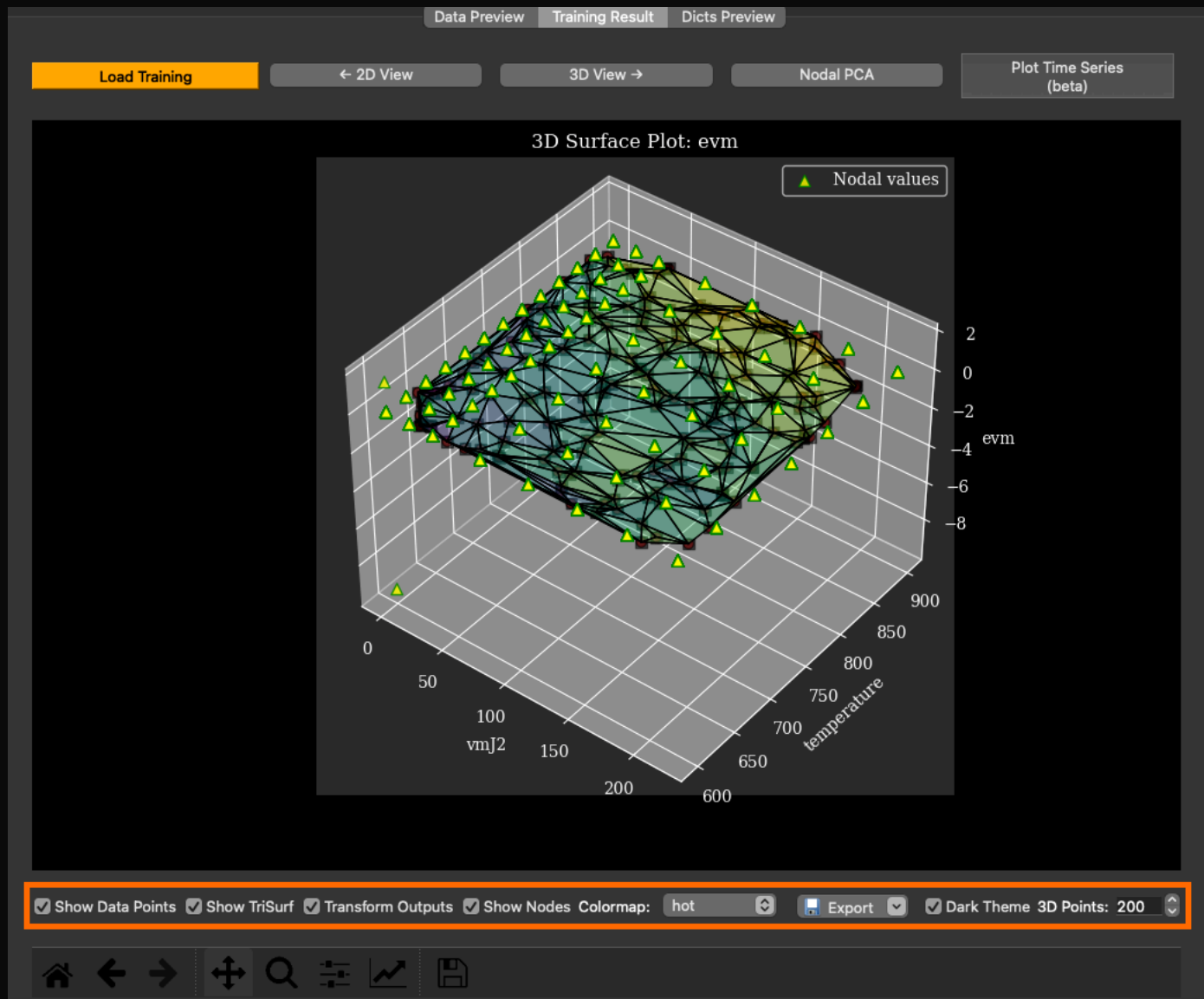
Visualization Controls

Controls available for result visualization:

- **Show Data Points:** Toggle visibility of training data points
- **Show TriSurf:** Toggle 3D surface plot. This plot is a guide for the eye to represent the surrogate response surface interpolated through the data point. (Note: the TriSurf is not a true representation of the actual shape functions in between nodes).
- **Transform Outputs:** Show results in mapped or physical space
- **Show Nodes:** Toggle visibility of mesh nodes (yellow triangles)
- **Colormap:** Select a colormap for the visualization

- **3D Points:** Control the number of points shown in 3D view

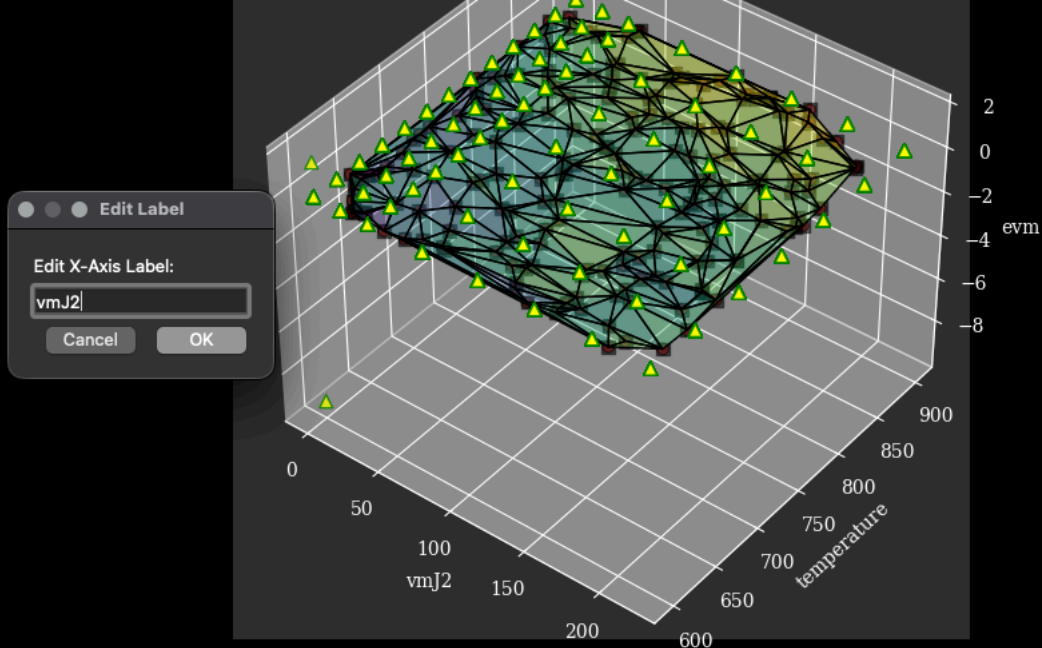
Conbtrols are located in the bottom banner below the graphs. For example, the nodal locations (yellow triangles) can be added/removed in the graph.



Configure Graphs

Axis Labels

The user can manipulate the axes in each Training Result graph to reflect the correct variable name or math symbol by a single-click on the axis label.



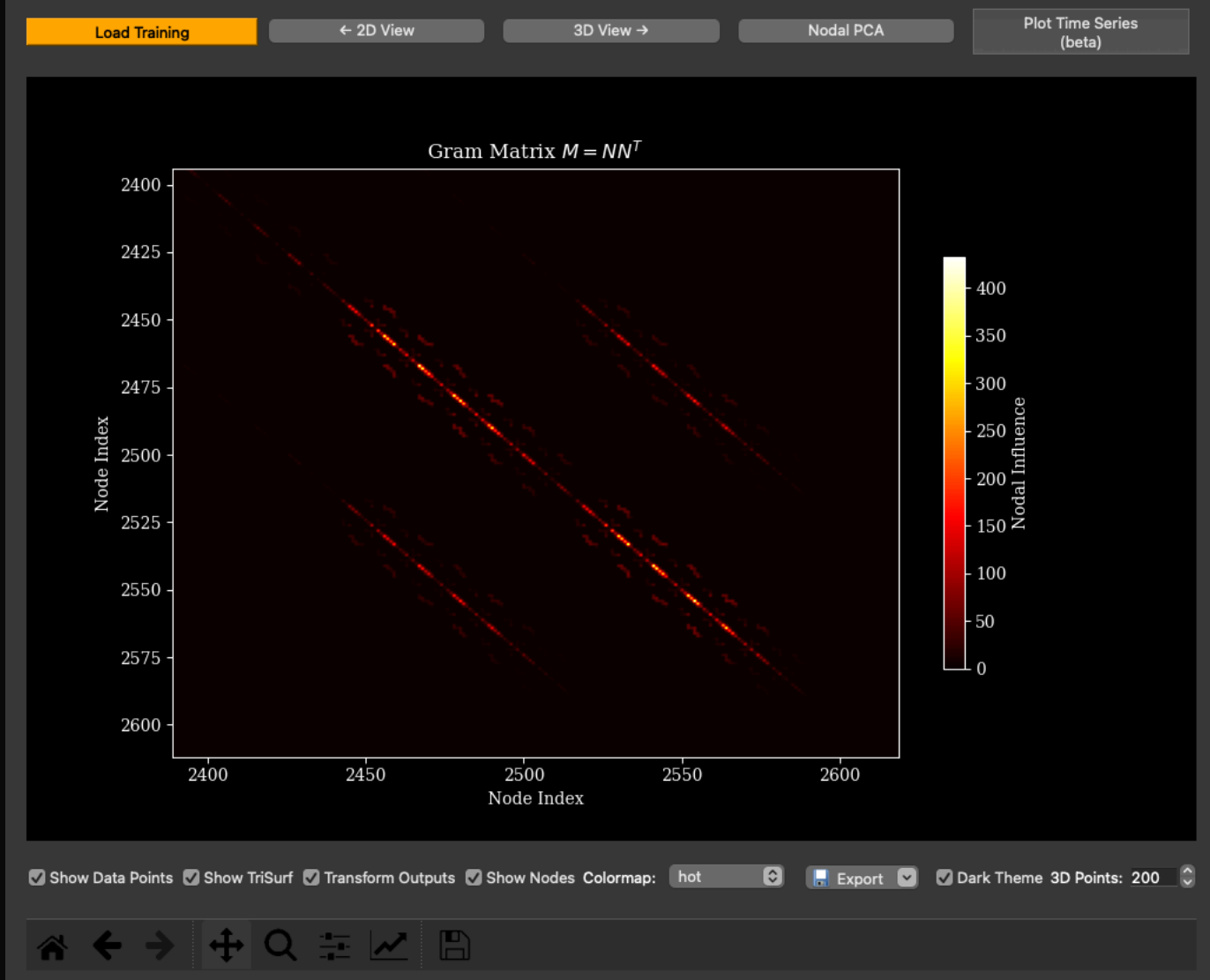
Axis Label Manipulation

PCA Analysis (beta)

Principal Component Analysis helps understand nodal relationships:

1. Click the **Nodal PCA** button to generate a heatmap.
2. The heatmap shows the Gram matrix ($M = NN^T$) representing nodal influence.

Training Results: Gram Matrix Exploration



This visualization shows the sparsity pattern of the Gram matrix (the normal-equations matrix) used when solving for the nodal values. The matrix is of size $N_{nodes} \times N_{nodes}$.

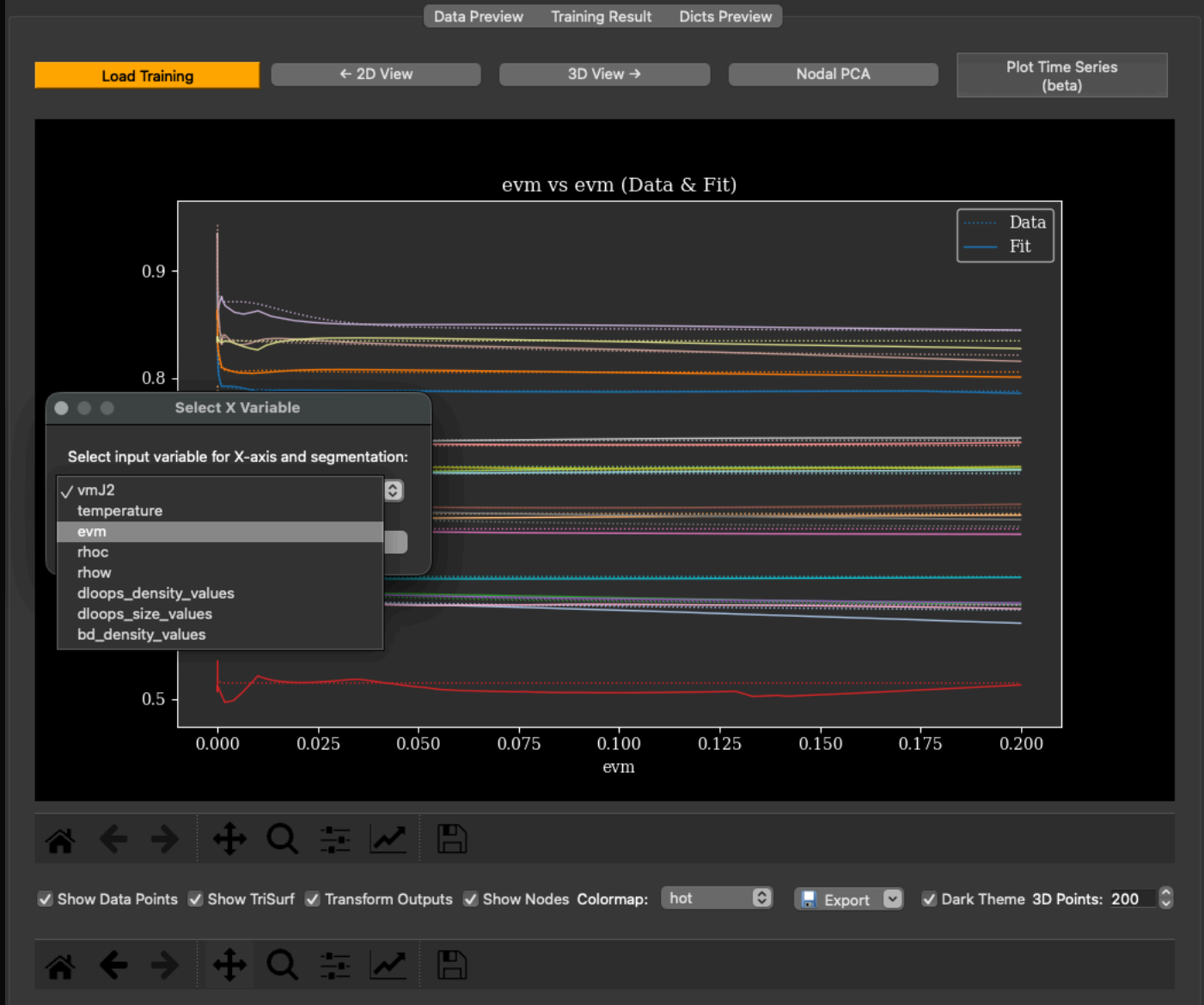
Black areas represent zero (or near-zero) entries, meaning those pairs of nodes have no overlapping support — their shape functions do not interact. Brighter regions indicate where neighboring nodes overlap.

Because the Lagrangian shape functions are strictly local, the matrix is extremely sparse when the number of nodes is reasonably large relative to the range of the input data. The more nodes you place across the same data range, the smaller each node's region of influence becomes, and the blacker (sparser) the heatmap becomes.

This extreme locality is the key advantage of the method: it allows efficient use of sparse linear solvers even for thousands of nodes. Use the zoom and pan tools to explore the local band structure and verify how compact the support of each nodal function really is.

Time Series Analysis

The **Plot Time Series (beta)** button provide functionality to visualize an output variable versus a selected input, optionally overlaying a surrogate-model fit, by first deserialising the data that were originally stored in a pickle file. The method retrieves the numeric arrays from either the class's `self.data` or a supplied DATA dict, which already contain the deserialized NumPy data. If the dataset includes `meta_data` with a `sim_lengths` field, the function uses this information—recorded at serialization time—to split the series into distinct simulation segments that represent consecutive moments in time. When such metadata are absent, it falls back to detecting large negative jumps in the X-axis values (indicative of resets) to heuristically segment the data, thereby approximating consecutive model responses. After cleaning, aligning lengths, and removing non-finite points, each segment is plotted (suspicious non-monotonic segments are highlighted in red) and embedded into the Qt interface, providing an interactive time-series view that respects the original data's temporal structure.



Time Series Graph

For time-dependent data:

1. Click the **Plot Time Series (beta)** button.
2. Select the input variable to use as the x-axis, and which correlates with the one-directional progression of time. For creep mechanics problems, this might be the strain.
3. The plot shows data and surrogate model predictions over time.

Export and Save Options

Saving the Surrogate Model & Training Results

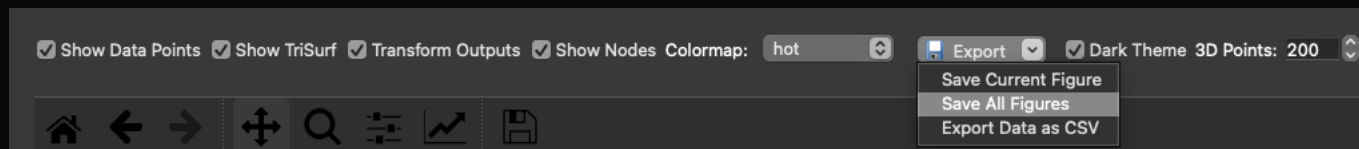
The outputs of the Surrogate Training Module are automatically saved in `../sm_wordir/GUIDemo` when training has successfully completed, and will be stored in the form of two Python pickle files:

1. `SM.pickle` – a Python-pickled dictionary that stores the trained surrogate model. The dictionary has the following keys:
 - `input_maps`: scikit-learn-style scaler objects used to map the raw input variables to the scaled space.
 - `output_maps`: scaler objects for the output variables.
 - `roi`: a dictionary describing the region-of-interest for each variable.
 - `mesh`: the most important entry; it holds the nodal coordinates and the connectivity matrix defining the interpolation mesh.
 - `mesh_specs`: specifications used to build the mesh (element counts, extrusion settings, etc.).
 - `nodes_mapped`: boolean indicating if nodal values are scaled or in original physical units.
 - `nodal_values`: the calibrated nodal values (physical units of output response) obtained after training.
 - `constrained`: a boolean indicating whether output constraints were applied.
 - `data_file`: path to the original data file that was used for training.

2. `train_output.pickle`, which contains the predictions at the trained inference points.

Export Plots

After the user has configured the Training Results plots, the plots can be exported by clicking the Export button:



Export Training Results

To export visualizations:

1. Click the **Export** button in the results panel.
2. A new prompt will appear with some options for saving and naming figures or .csv files. Choose from:
 - **Save Current Figure**: Save the currently displayed figure
 - **Save All Figures**: Save all available figures
 - **Export Data as CSV**: Export plot data as a CSV file

When saving figures, you can configure:

- Title and filename
- Resolution (DPI)
- Size adjustment
- Background transparency
- Theme (current, dark, or light)
- Custom axis labels

Theme Customization

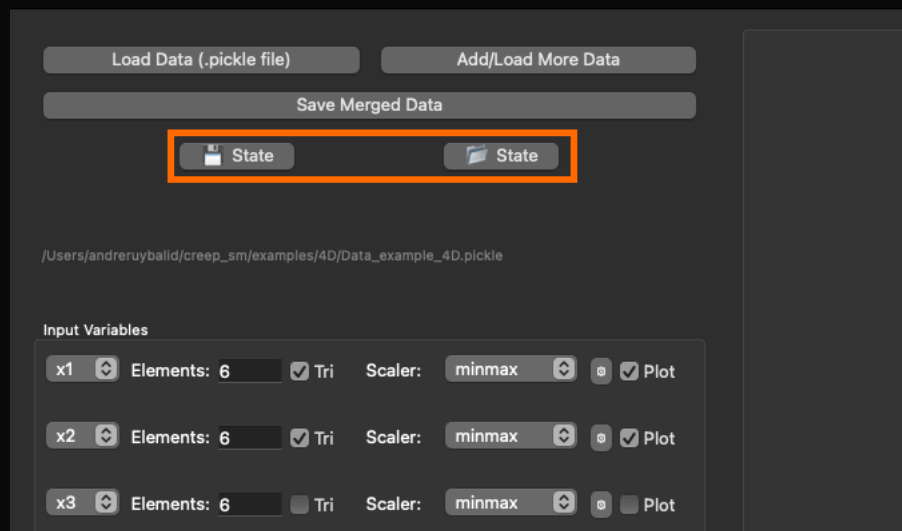
Use the **Dark Theme** checkbox to toggle between dark and light themes for all plots.

Custom Colormap Selection

Select from various colormaps (hot, viridis, plasma, etc.) to customize visualizations.

Additional Notes

- For very large datasets, use the preview points control to improve performance
- The application automatically saves your session state to `last_gui_state.json`
- You can save custom training configuration "State" .json files for different projects. These .json state files can be saved and opened with the buttons "Load State" and "Save State":



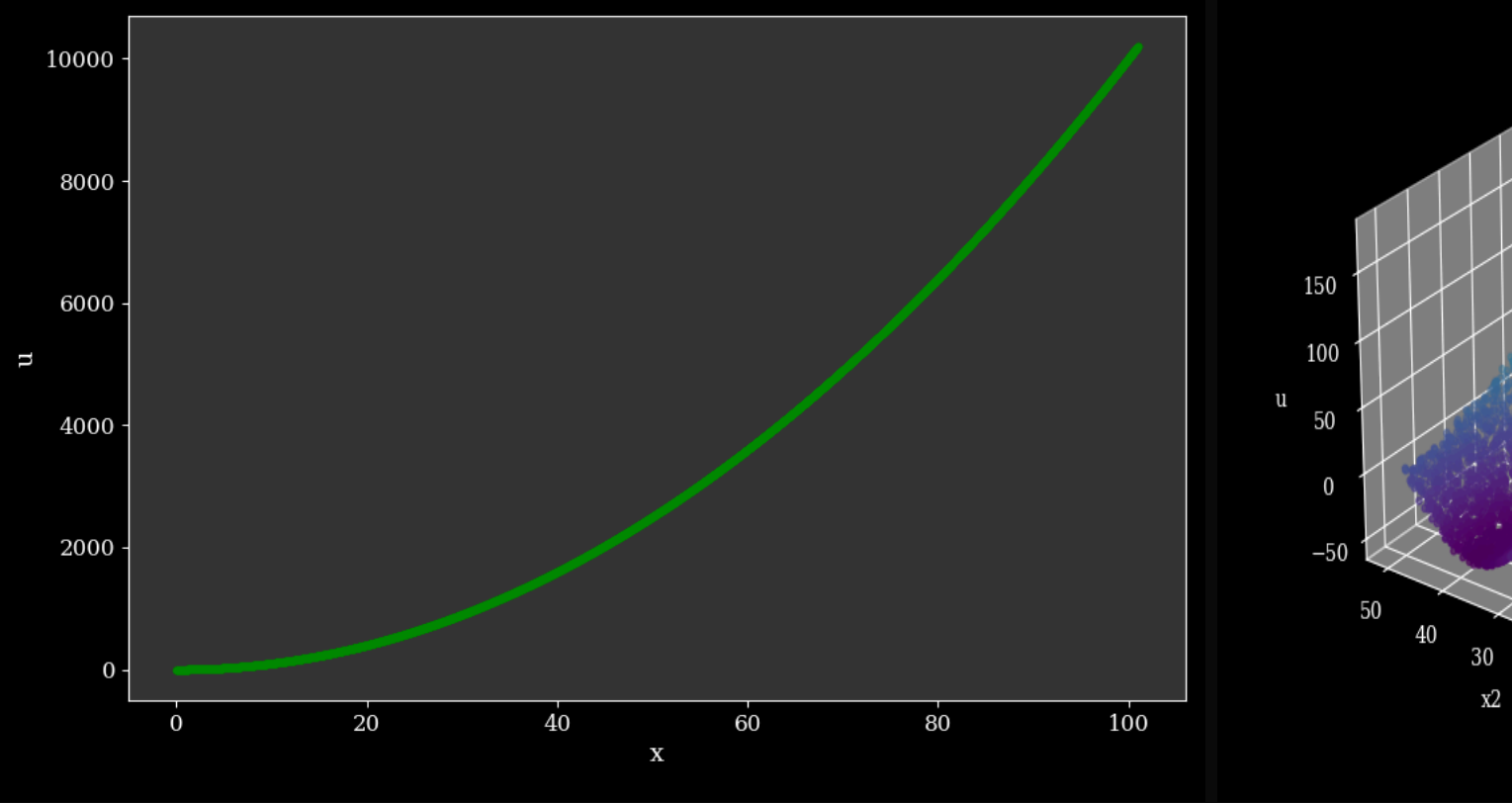
Load Training Configuration state files.

When loading in a previously saved state file (.json) containing training configurations, it is still needed to rebuild the mesh. The user must click the "Preview Mesh" button after the state-file has been loaded, otherwise, upon training ("Run Training"), the mesh will be unknown and the following error will be printed in the console:

```
line 178, in sm_train
    ndim_base = np.array(mesh_specs['premade_mesh']['nodes']).shape[1]
IndexError: tuple index out of range
Abort trap: 6
```

Examples

Example data is located the `examples` directory, containing examples of 1D, 2D, 3D, 4D, and 8D datasets. These can be used to test the Training Module and to learn its features. Important note: the mesh preview functionality will not work with the 1D data, because it expects to build a 2D mesh preview.



Screenshots of Data Examples in the `examples` folder. From left to right: 1D, 2D, 4D datasets.

There are four different examples in the `examples` directory, each with a different number of datapoints and dimensions. The dimensionality of the input space is used to name the examples, i.e., "1D", "2D", "4D", "8D".

Example	Description	Note
1D	quadratic curve	simple line fit
2D	smooth peaks and valleys	hyperdimensional surface fit
4D	sharp flattened peak	sharp variations to capture: try mesh refinement
8D	realistic creep data	hyperdimensionality

Use the .json state files in the `examples` directory to load in training configurations that work with the corresponding data. Then recreate similar results from scratch to understand the influence of all the settings.

Backend Logic Overview

The core modules reside in the `lib` directory, which should be placed alongside `training_module.py`.

Core Modules

- **fe_shapes.py** – Implements mixed simplex/hypercube shape functions, local interpolation matrix construction, and global sparse interpolation matrix assembly (`fe_shapefunc_sparse_mixed`). Handles element lookup (`find_element_mixed`), barycentric coordinate calculation, and permutation to align local node ordering with global connectivity.
- **sm_data.py** – Provides data loading, preprocessing, and scaling utilities. Constructs the mesh specification dictionary (`mesh_specs`) containing element numbers, simplex dimensions, and distribution parameters.
- **sm_build.py** – Orchestrates mesh generation (`build_simplex_extrusion_nd`), refinement (`refine_lasso_region`), and element removal. Calls `fe_shapes` for shape evaluation and assembles the node/element arrays.
- **sm_analyze.py** – Post-processing tools for error metrics, PCA analysis, and diagnostics of the assembled matrix (`N`). Includes functions to compute relative error, residuals, and sparsity patterns.
- **sm_plot.py** – Visualization helpers for mesh preview, data plotting, and result rendering. Wraps Matplotlib calls used by the GUI.
- **sm_rtree.py** – Optional R-tree spatial index for accelerated element lookup in high dimensions. Provides `find_element_rtree` used by `fe_shapes.find_element_mixed`.

Key Algorithms and Mathematics

1. Mixed Shape-Function Construction (`fe_shapefunc_sparse_mixed`)

- **Dimension split:** Separates node coordinates into simplex dimensions (`tri_elements`) and hyper-cube (extrusion) dimensions.
- **Simplex part:** Computes barycentric coordinates via `map_to_barycentric_nd`, yielding shape values that satisfy the Kronecker-delta property on the reference simplex.
- **Hypercube part:** Generates tensor-product linear shape functions for each extrusion dimension by evaluating the product of 1-D hat functions at the normalized local coordinates.
- **Tensor product:** Forms the local interpolation matrix `N_elem = shape_simplex ⊗ shape_hypercube`.
- **Permutation:** Calculates a permutation array that maps the natural tensor-product ordering to the order of node indices in the connectivity matrix, ensuring correct assembly.
- **Sparse assembly:** For each training point, selects the element with the maximum shape-function value, stores the corresponding row in COO format, and finally builds a CSR matrix `N_sparse`.

2. Global Matrix Assembly

- The COO data (`dat`, `row`, `col`) are accumulated across all elements and points.
- The matrix `N_sparse` has size `N_points × N_nodes`. Its transpose-product `NTN` is symmetric positive-definite, enabling stable solution of the normal equations.

3. Mesh Generation (`build_simplex_extrusion_nd`)

- Creates a base 2-D simplex mesh using Delaunay triangulation (`scipy.spatial.Delaunay`).
- Extrudes the base mesh into higher dimensions by stacking copies along the extrusion axes, respecting `mesh_specs['element_numbers']`.
- Ensures proper orientation (`ensure_positive_orientation_nd`) and enforces checkerboard diagonals for 2-D elements.

4. Refinement (`refine_lasso_region`, `refine_lasso_region_2d`)

- Projects node coordinates onto the 2-D base plane.
- Generates new nodes by bisecting grid lines within the selected lasso polygon.
- Lifts new nodes back into the full dimensional space, preserving existing coordinate values in extrusion dimensions.

5. Data Density Filtering (`data_density_per_element_mixed`)

- Counts training points per element using fast bounding-box rejection followed by precise point-in-element tests.
- Flags sparse elements for removal or refinement based on a user-defined threshold.

6. Solver (`NTN` system)

- After assembling `N`, the backend solves $w = (NTN)^{-1} NT^T y$ using SciPy sparse linear solvers (`scipy.sparse.linalg.spsolve` or conjugate-gradient).
- Regularization options are available in `sm_analyze.py` to improve conditioning.

Debugging & Optimization Guide

Area	Typical Bottleneck	Debug Tips	Optimization Strategies
Element lookup (<code>find_element_mixed</code>)	High dimensionality leads to many candidates	Enable <code>verbose=True</code> to print bounding-box stats; compare R-tree vs brute-force timings	Use R-tree (<code>sm_rtree</code>) or pre-compute element bounding boxes

Area	Typical Bottleneck	Debug Tips	Optimization Strategies
Shape-function evaluation	Repeated barycentric solves	Profile <code>map_to_barycentric_nd</code> ; cache <code>verts_simplex</code> per element	Vectorise barycentric computation; use NumPy <code>lstsq</code> only once per element
Sparse assembly	Large COO arrays consume memory	Inspect <code>dat</code> length; check for duplicate rows	Incrementally build CSR using <code>scipy.sparse.lil_matrix</code> for very large datasets
Mesh refinement	Excessive node duplication	Verify <code>refine_lasso_region</code> does not create near-duplicate nodes (check tolerance)	Increase <code>spacing</code> tolerance; deduplicate with <code>np.unique</code> after refinement

Where to Find the Code

- **Shape functions & assembly** – [lib/fe_shapes.py](#), functions: `fe_shapefunc_sparse_mixed`, `map_to_barycentric_nd`, `find_element_mixed`.
- **Mesh creation** – [lib/sm_build.py](#), functions: `build_simplex_extrusion_nd`, `refine_lasso_region`, `generate_refinement_nodes`.
- **Data handling** – [lib/sm_data.py](#), functions: `load_data`, `scale_variables`, `prepare_mesh_specs`.
- **Analysis utilities** – [lib/sm_analyze.py](#), functions: `compute_error_metrics`, `pca_nodal`, `sparsity_report`.
- **Visualization** – [lib/sm_plot.py](#), functions: `plot_mesh`, `plot_results`, `interactive_lasso`.
- **Publishing** – [lib/sm_publish.py](#), functions: `export_figure`, `save_state`, `load_state`.
- **Spatial indexing (optional)** – [lib/sm_rtree.py](#), class `MeshRtreeIndex`, method `find_element_rtree`.

These modules together constitute the backend logic that powers the GUI. Understanding the data flow—`sm_data` → `sm_build` → `fe_shapes` → `sm_analyze` → `sm_publish`—will help you locate performance hotspots and debug issues effectively.

Troubleshooting

Common Issues

- "No Data" warning:**
 - Ensure you've loaded a valid dataset
 - Check that the file has the expected structure
- "Input Selection" warning:**
 - Select 1-2 input variables for plotting by checking their "Plot" boxes
- "No Output Selected" warning:**
 - Select an output variable for plotting by checking the "Plot" box
- "No ROI" warning:**
 - Draw an ROI on the plot before previewing the mesh or training
- Poor mesh quality:**
 - Adjust element numbers for each dimension
 - Use the **Refine Region** tool to add more elements in important areas
 - Remove elements with low data density
- High relative errors:**
 - Try different scaling methods for inputs and outputs
 - Add constraints to bound output values
 - Increase mesh resolution in areas with high gradients

Error Messages

- **"Invalid Elements"**: Element inputs must be integers or lists of numbers
- **"Insufficient variables"**: Select at least one input and one output
- **"Segmentation Warning"**: Non-monotonic x-variable in time series plot

For additional issues, inspect the Python console output for detailed error messages.

Surrogate Model Overview

The **LAROMance** surrogate modeling framework provides a mathematically rigorous, finite-element-inspired approximation of high-dimensional physical responses. Below each component of the formulation is presented together with a clear definition of every symbol used.

1. Problem Setting and Dimensionality

- **Input variables** – Denoted $x_1, x_2, \dots, x_{d-1}$, these are the independent parameters supplied by the user (e.g., temperature, stress, dislocation density, strain history, etc.)
- **Output variable** – Denoted y , this is the single scalar quantity the surrogate predicts (e.g., a specific stress-strain measure).
- **Total dimension** – $d = (d - 1) + 1$. The surrogate operates in a d -dimensional Euclidean space $\mathbb{R}^d$, where the first $d - 1$ dimensions correspond to the inputs and the final dimension corresponds to the output. In the creep surrogate model applications relevant for this type of surrogate, the dimensionality is typically $d > 3$, meaning the method works in three or more dimensions (tested up to 8 dimensions successfully) and is dimension-agnostic in the sense that it can train across any dimensionality of a given dataset.
- **Physical domain** – The domain $\Omega \subset \mathbb{R}^d$ is a Cartesian hyper-rectangle defined by the admissible ranges of all variables. For each input x_j we have lower and upper bounds $x_j^{\min}$ and $x_j^{\max}$, and for the output y we may also prescribe bounds if physical limits are known.

2. Mesh Discretisation

The input sub-domain (the first $d - 1$ dimensions) is partitioned into a structured mesh consisting of **elements** e and **nodes** i :

- **Elements** – Each element e may be a d -dimensional simplex (e.g., a triangle in 2-D) or a hyper-rectangle. The choice is controlled by the *Element type* setting in the GUI.
- **Nodes** – The geometric corners of the elements. For each node i we store a **nodal coordinate vector**

$$\mathbf{x}_i = (x_{i,1}, x_{i,2}, \dots, x_{i,d-1}, w_i)^\top \in \mathbb{R}^d.$$

The first $d - 1$ components are the prescribed input values, while the last component w_i is the *nodal output value* that will be solved for during training.

3. Reference Element and Isoparametric Mapping

All physical elements are mapped from a **reference element** $\hat{\Omega} = [-1, 1]^d$ using **isoparametric coordinates**

$$\boldsymbol{\xi} = (\xi_1, \xi_2, \dots, \xi_d) \in \hat{\Omega}.$$

The same shape functions that interpolate the field are used to map geometry, guaranteeing that the interpolation of geometry and of the surrogate field are consistent.

4. Shape-Function Interpolants

- **One-dimensional Lagrange shape functions** $\ell_{ij}^{(j)}(\xi_j)$ of polynomial degree p satisfy the Kronecker-delta property

$$\ell_{ij}^{(j)}(\xi_{mj}) = \delta_{ij} m_j, \quad m_j = 0, 1, \dots, p.$$

Here i_j indexes the local node position along the j^{th} direction and δ is the Kronecker delta (1 if $i_j = m_j$, 0 otherwise).

- **Multidimensional shape functions** are obtained by a tensor product of the one-dimensional functions:

$$\phi_{\mathbf{i}}(\boldsymbol{\xi}) = \prod_{j=1}^d \ell_{i_j}^{(j)}(\xi_j).$$

The multi-index $\mathbf{i} = (i_1, i_2, \dots, i_d)$ uniquely identifies a node in the reference element.

For the common bilinear case $d = 2$ and $p = 1$, the four shape functions are

$$\begin{aligned} \phi_{(0,0)}(\xi_1, \xi_2) &= (1 - \xi_1)(1 - \xi_2), \\ \phi_{(1,0)}(\xi_1, \xi_2) &= \xi_1(1 - \xi_2), \\ \phi_{(0,1)}(\xi_1, \xi_2) &= (1 - \xi_1)\xi_2, \\ \phi_{(1,1)}(\xi_1, \xi_2) &= \xi_1\xi_2. \end{aligned}$$

5. Local Interpolation Matrix

Given a set of training data points $\{\mathbf{x}_{\text{data}}^{(k)}\}_{k=1}^N$ that lie inside element e , each point is first mapped to reference coordinates $\boldsymbol{\xi}^{(k)}$. The **local interpolation matrix** $\mathbf{N}^{(e)} \in \mathbb{R}^{N_e \times n_e}$ (where N_e is the number of training points in element e and n_e the number of local nodes) has entries

$$\mathbf{N}_{k,i}^{(e)} = \phi_{\mathbf{i}}(\boldsymbol{\xi}^{(k)}).$$

- k indexes the training points (rows).
- i indexes the local nodes of element e (columns).

Each row therefore contains the evaluated shape functions at a particular training point, providing the weighting coefficients that blend the nodal output values to reproduce the observed output.

6. Global Interpolation Matrix

All element matrices are assembled into a **global interpolation matrix** $\mathbf{N} \in \mathbb{R}^{N \times M}$, where N is the total number of training points and M the total number of nodes in the mesh.

The assembly is performed by the routine `fe_shapefunc_sparse_mixed` in `lib/fe_shapes.py`. For each element the algorithm:

1. **Extracts node coordinates** and separates them into simplex (triangular) dimensions and hyper-cube (extruded) dimensions according to the mesh specification (`mesh_specs['tri_elements']`).
2. **Evaluates shape functions** – simplex shape functions are obtained via barycentric coordinates (`map_to_barycentric_nd`), while hyper-cube shape functions are built as a tensor product of 1-D linear functions.
3. **Builds the local interpolation matrix** $\mathbf{N}^{(e)}$ by taking the outer product of the simplex and hyper-cube shape vectors.
4. **Computes a permutation** that maps the natural tensor-product column ordering to the order of node indices stored in the connectivity matrix, ensuring that each column of $\mathbf{N}^{(e)}$ aligns with the correct global node.
5. **Inserts the permuted rows** into a COO sparse matrix, which is later converted to CSR format to obtain the global $\mathbf{N}$.

Because the same connectivity rules as traditional FEM are used, the resulting global matrix is sparse and enforces $\mathcal{C}^0$ continuity across element boundaries, even in dimensions higher than three. The mixed-element approach — using a 2-D simplex base extruded into additional dimensions (i.e., a prism in 3D) — is not standard in classical FEM, but it yields a lightweight custom element that preserves proper bookkeeping of the connectivity and nodal coordinate matrices while supporting >3D problems.

7. Linear System for Nodal Output Values

Let $\mathbf{y} \in \mathbb{R}^N$ denote the vector of known output values from the training data. The surrogate seeks nodal output coefficients $\mathbf{w} \in \mathbb{R}^M$ such that

$$\mathbf{N} \mathbf{w} \approx \mathbf{y}.$$

Because $\mathbf{N}$ is typically rectangular (more data points than nodes), we solve the **normal equations**

$$(\mathbf{N}^\top \mathbf{N}) \mathbf{w} = \mathbf{N}^\top \mathbf{y},$$

or equivalently

$$\mathbf{w} = (\mathbf{N}^\top \mathbf{N})^{-1} \mathbf{N}^\top \mathbf{y}.$$

The matrix $\mathbf{N}^\top \mathbf{N}$ is symmetric positive-definite and sparse, which permits the use of efficient sparse linear solvers.

8. Prediction Formula

After solving for $\mathbf{w}$, evaluating the surrogate at any new input point $\mathbf{x} \in \Omega$ proceeds as follows:

1. **Locate the element** e that contains $\mathbf{x}$.
2. **Map $\mathbf{x}$ to reference coordinates** $\boldsymbol{\xi}$ of element e using the isoparametric mapping.
3. **Evaluate the shape functions** $\phi_i(\boldsymbol{\xi})$ for all nodes of e .
4. **Form the local interpolation vector** $\mathbf{N}^{(e)}(\mathbf{x}) = [\phi_i(\boldsymbol{\xi})]_i$.
5. **Compute the surrogate output**

$$y(\mathbf{x}) = \mathbf{N}^{(e)}(\mathbf{x}) \mathbf{w}^e,$$

where $\mathbf{w}^e$ denotes the subset of nodal coefficients associated with element e .

9. Practical Considerations

- **Variable scaling and transformation** – Prior to mesh generation each input variable is typically **scaled** to a common interval (e.g., $[-1, 1]$). Non-linear transformations (logarithmic, power-law) may also be applied to improve the linearity of the input-output relationship and to reduce conditioning issues in $\mathbf{N}^\top \mathbf{N}$.
- **Initial nodal placement** – The mesh is initially generated as a regular Cartesian grid that uniformly covers the full input domain. This provides a baseline resolution and ensures that every region of the domain is represented before any refinement.
- **Error metrics and validation** – Surrogate quality is quantified using:
 - **Relative L_2 error**

$$\varepsilon_{\text{rel}} = \frac{\|\mathbf{y} - \hat{\mathbf{y}}\|_2}{\|\mathbf{y}\|_2},$$

- **Maximum absolute error** $\max_k |r_k|$,
- **Cross-validation** or hold-out test sets, which are displayed in the “Results Analysis” tab.
- **Boundary handling** – Nodes that lie on the physical domain boundary may be assigned Dirichlet conditions if the user provides known output values at those boundaries; otherwise they remain free unknowns and are solved together with interior nodes.
- **Hyper-parameters** – The user can control:
 - **Polynomial degree** p of the shape functions (higher p yields smoother approximations at the cost of larger element matrices).
 - **Initial element count per dimension** (coarse vs. fine mesh).
 - **Refinement criteria** (error tolerance, maximum refinement depth).

10. Summary of Symbols

Symbol	Meaning
d	Total number of dimensions (inputs+output).
p	Polynomial degree of the Lagrange shape functions (commonly $p = 1$ for multilinear interpolation, $p > 1$ for higher-order).
$\mathbf{x}_i$	Nodal coordinate vector at node i , containing all input variables $x_{i,1}, \dots, x_{i,d-1}$ and the unknown output value w_i .
$\mathbf{y}$	Vector of known output values from the training data set (size N).
$\mathbf{N}^{(e)}$	Local interpolation matrix for element e , with entries $\phi_{\mathbf{i}}(\boldsymbol{\xi}^{(k)})$.
$\mathbf{N}$	Global interpolation matrix assembled from all $\mathbf{N}^{(e)}$, size $N \times M$, sparse, enforcing $\mathcal{C}^0$ continuity.
$\mathbf{w}$	Vector of unknown nodal output coefficients (size M) solved via the normal equations. These nodal values are the degrees of freedom to the problem.
$\phi_{\mathbf{i}}$	Multidimensional shape function associated with node index $\mathbf{i}$.
$\ell_{ij}^{(j)}$	One-dimensional Lagrange shape function in direction j , of degree p .
$\hat{\Omega}$	Reference element $[-1, 1]^d$, used for isoparametric mapping.
$\boldsymbol{\xi}$	Isoparametric coordinates in the reference element.
$\mathbf{y}(\mathbf{x})$	Predicted surrogate output for a new input point $\mathbf{x}$.
e	Index of the element that contains the evaluation point $\mathbf{x}$.
r_k	Residual error at training point k , $r_k = y_k - \hat{y}_k$.
ϵ_{rel}	Relative L_2 error metric for surrogate validation.

The **LAROMance** surrogate provides a mathematically rigorous, FEM-inspired approximation across a hyperdimensional input space (>3D), that can be evaluated instantly after the offline training phase, while offering the flexibility to adapt mesh resolution, nodal distribution, polynomial degree, and variable scaling to capture highly non-linear behaviour in complex, high-dimensional physical systems.